

Redes TCP/IP - Capa de Enlace

Redes de Comunicación, Manipulación y Telemedicina
Departamento de Ingeniería de Sistemas y Automática



Escola de
Enxeñaría
Industrial
Universidade de Vigo

Contenido

- Introducción a la capa de enlace
- Delimitación de trama
- Tratamiento de errores
- Detección de errores
- Corrección de errores
- Retransmisiones
- Control de acceso al medio
- LAN conmutado

Introducción a la capa enlace

Modelo TCP/IP



Modelo de Referencia TCP/IP
4 capas

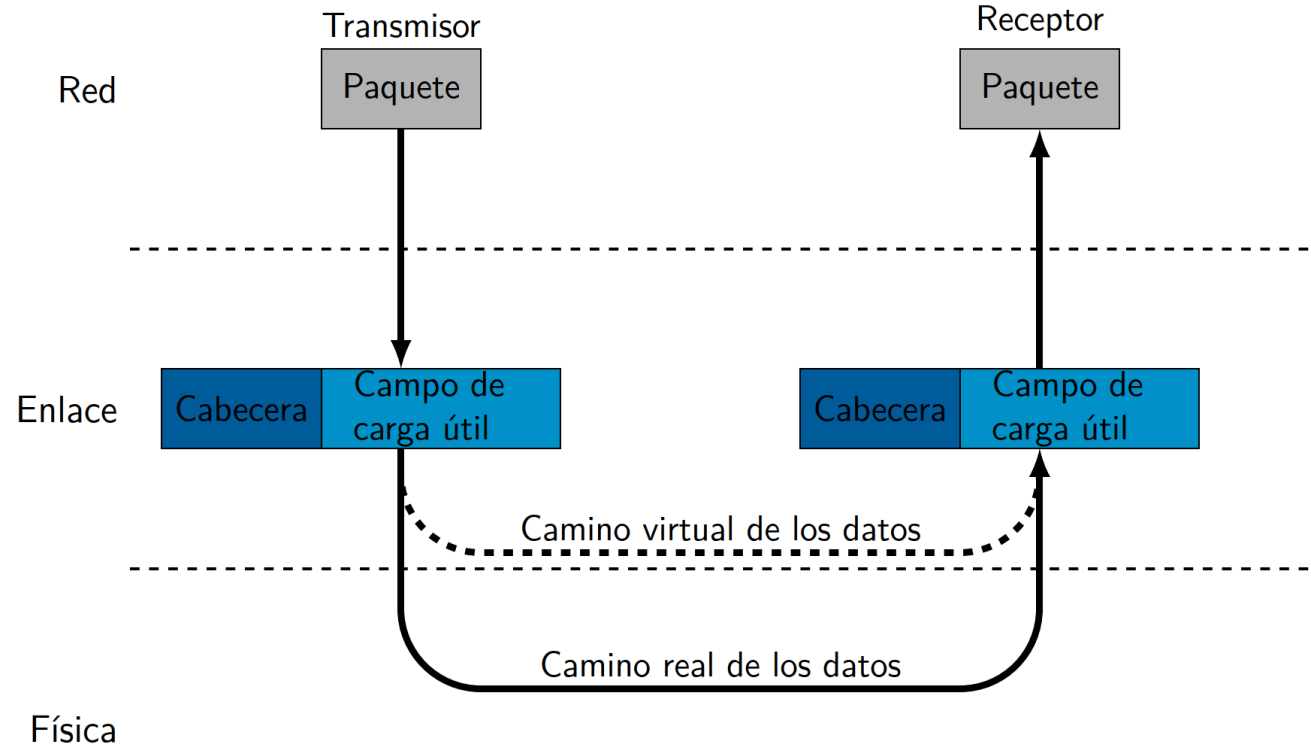


Modelo de Referencia TCP/IP
5 capas

Introducción

La capa de enlace es responsable de transferir paquetes a través de un enlace físico. Estos paquetes, llamados tramas, tienen un tamaño limitado y deben organizarse adecuadamente para garantizar una transmisión eficiente. Para cumplir su función, la capa de enlace utiliza los servicios de la capa física, que se encarga de enviar los bits a través del canal de comunicación.

Entre los principales servicios que ofrece la capa de enlace se encuentran la estructura y delimitación de tramas que organiza los datos para su transmisión. También se asegura de que los datos se entreguen de manera fiable mediante la detección y corrección de errores. Si hay fallos en la transmisión, se pueden solicitar retransmisiones automáticas (Automatic Repeat reQuest, ARQ). Por último, la capa de enlace gestiona el control de acceso al medio compartido y permite la conmutación a nivel de enlace, facilitando la comunicación en redes más complejas.



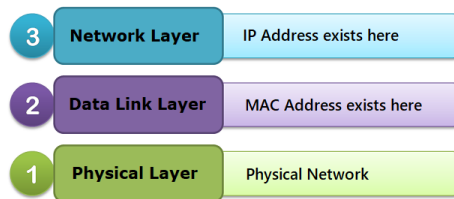
Delimitación de trama

Las direcciones MAC (Media Access Control) son identificadores físicos asignados a las tarjetas de red y son esenciales para la comunicación en una red a nivel de hardware. Están grabadas físicamente en un chip ROM dentro de la tarjeta. Funcionan en la Capa de Enlace de Datos. Estas direcciones están formadas por seis bytes representados en formato hexadecimal. Dado que las direcciones MAC tienen una longitud de 48 bits, el sistema permite generar aproximadamente 16.7 millones de direcciones únicas por cada OUI. Cuando se agota las direcciones asignadas a un OUI, el fabricante solicita otro.

Una dirección MAC tiene dos partes. Los primeros tres bytes, o 24 bits, son asignados por el IEEE a los fabricantes de hardware. Esta sección de la dirección se conoce como el Identificador de Organización Única (Organizationally Unique Identifier, OUI). Gracias a este identificador, es posible reconocer de qué fabricante proviene un dispositivo, como por ejemplo Dell o HP. La segunda parte de la dirección, también de 24 bits, es asignada de manera secuencial por el fabricante. Esto garantiza que cada dispositivo producido tenga una dirección MAC única.

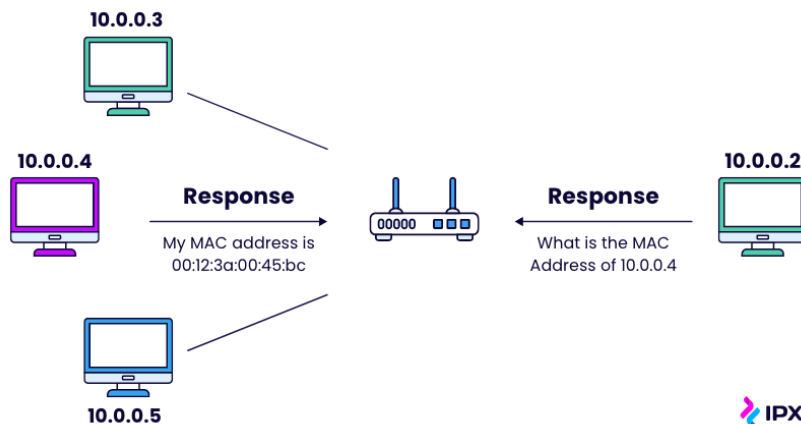
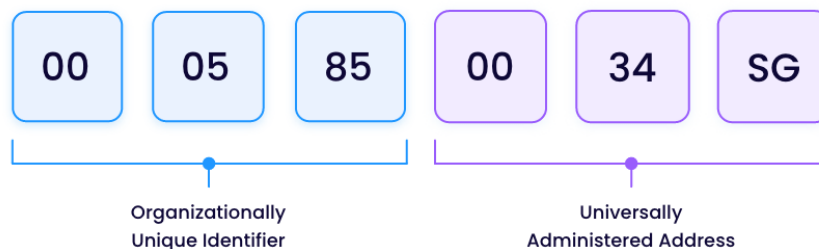
Por ejemplo, en la dirección MAC 00:21:70:6f:06:f2, los primeros tres bytes (00:21:70) indican el fabricante, mientras que los últimos tres bytes (6f:06:f2) identifican específicamente al dispositivo dentro de los productos de ese fabricante.

Es importante señalar que, aunque es posible modificar la dirección MAC a nivel del sistema operativo mediante un proceso llamado "suplantación" o "spoofing", la dirección real en la tarjeta física no cambia. En este caso, lo que cambia es cómo el sistema operativo maneja la dirección en su memoria, pero la dirección grabada en la tarjeta sigue siendo la misma. Esta técnica de suplantación es algo que se estudia en ciberseguridad como parte de los métodos de ataque.



TecAdmin.net

Media Access Control (MAC)

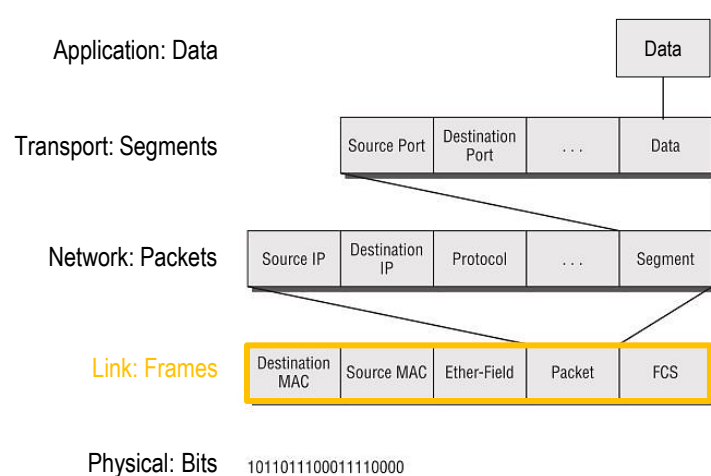


El formato de trama Ethernet II sigue una estructura específica para asegurar una transmisión eficiente y fiable. Las tramas deben tener una estructura inequívoca. Esto garantiza, salvo error, su correcto procesamiento por las capas superiores.

El proceso comienza con un preámbulo, una secuencia de bits que permite sincronizar los relojes del emisor y el receptor. Esta secuencia es 10101010 repetida siete veces, lo que facilita la preparación para recibir los datos.

Después del preámbulo, se encuentra la marca de inicio de trama o (Start Frame Delimiter, SFD) una secuencia especial de bits 10101011. A continuación, se incluyen las direcciones de la capa de enlace, tanto de origen como de destino, para identificar a los dispositivos que participan en la comunicación.

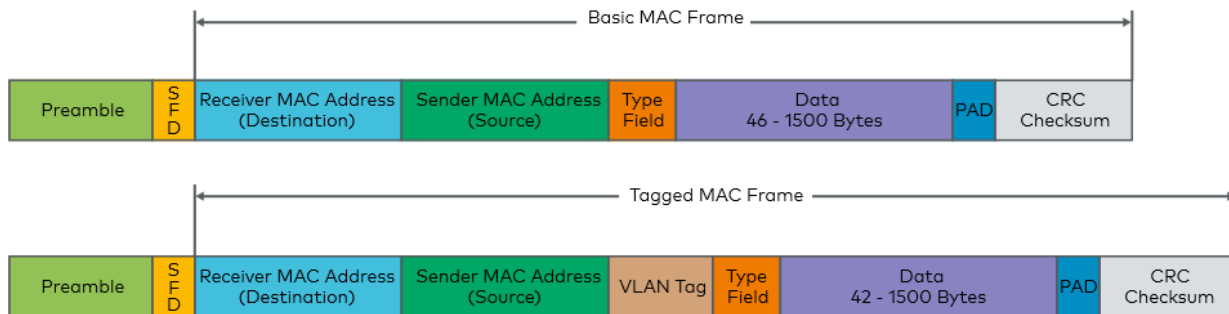
También se especifica el tipo de datos que se están transportando, seguido de los propios datos, cuya longitud es variable y depende de la capa superior que los genera. Finalmente, se encuentra la secuencia utilizada para detectar errores (Frame Check Sequence, FCS). La detección del final de la trama se realiza a nivel físico. Por ejemplo, cuando el receptor deja de recibir la señal portadora. Aunque en este formato no se muestran, podría haber otros campos opcionales.



Trama

Ethernet and IP

Ethernet II Frame



© 2010-2017, Vector Informatik GmbH. All rights reserved. Any distribution or copying is subject to prior written approval by Vector. V2.0

Entramado

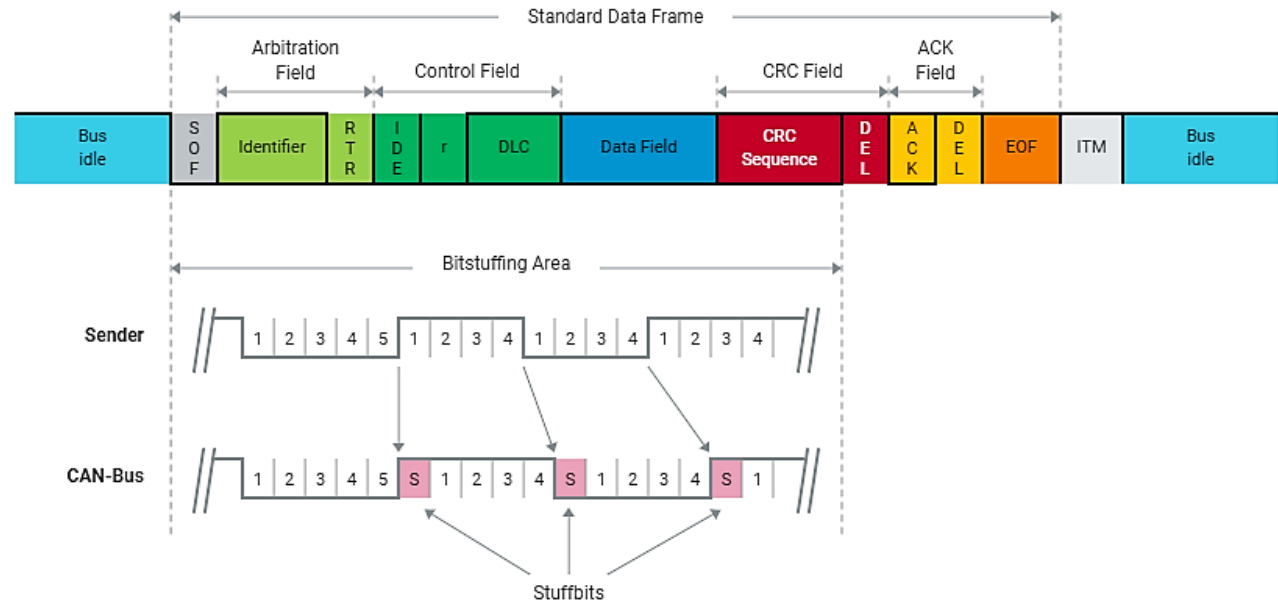
El entramado (bit stuffing) es el proceso mediante el cual se organizan los bits transmitidos por la capa física de una red en unidades llamadas tramas. La capa física se encarga de enviar una secuencia continua de bits. Es esencial delimitar las tramas. Hay que establecer con precisión dónde comienza y termina cada una. Esto evita confusiones en la recepción de los datos.

Existen varios métodos para delimitar las tramas: (1) cuenta de bytes, en la que se define la longitud de la trama en función del número de bytes, (2) uso de marcas y relleno de bytes, (3) uso de marcas y relleno de bits. En los casos (2) y (3), se emplean secuencias específicas que indican el inicio y fin de la trama, y si estas secuencias aparecen dentro de los datos, se añaden bits o bytes adicionales para diferenciarlas de los delimitadores.

Además de estos métodos, existen otras técnicas de delimitación que no se abordarán en esta asignatura.

CAN Framing

Bitstuffing



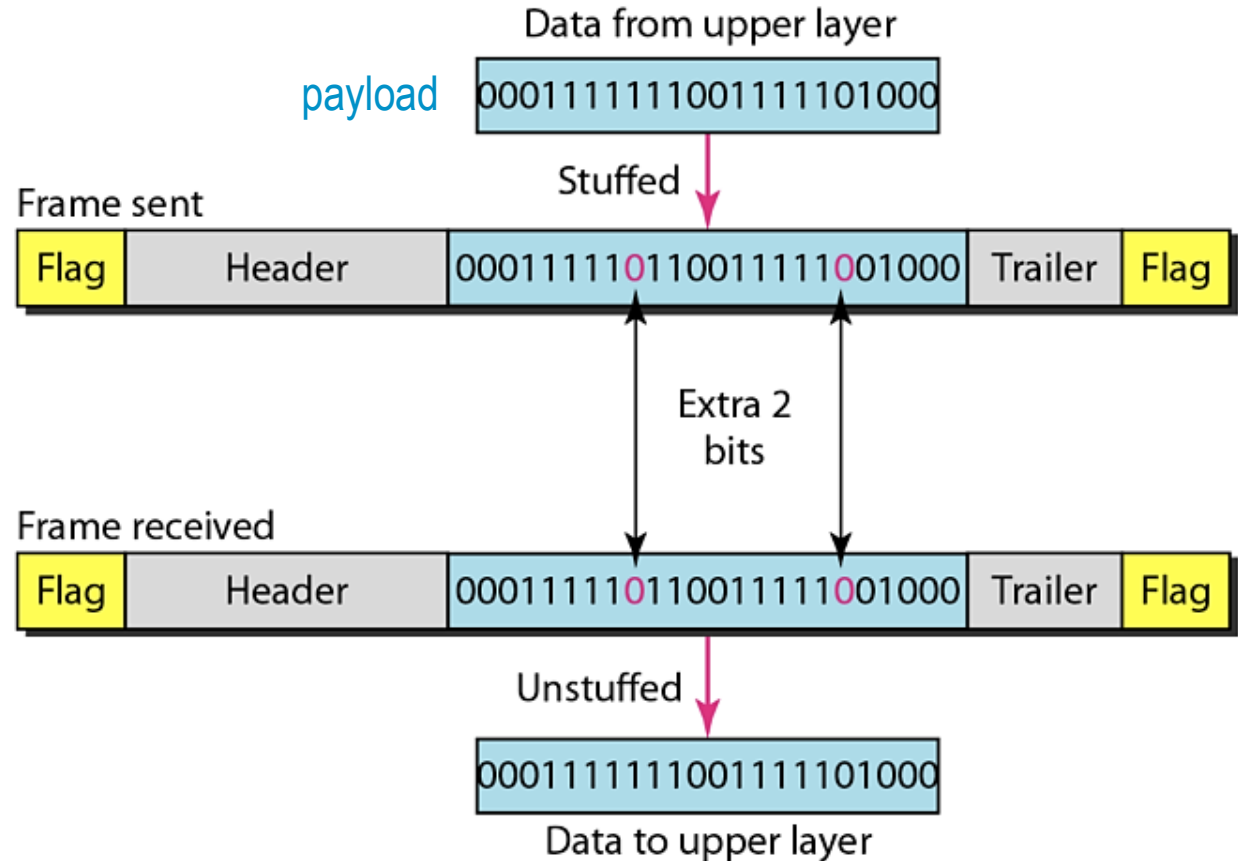
© 2010-2024, Vector Informatik GmbH. All rights reserved. Any distribution or copying is subject to prior written approval by Vector. V2.0

El método de marca y relleno de bits es una técnica de entramado que funciona a nivel de bits en lugar de bytes. En este caso, una secuencia específica de bits se utiliza para marcar el inicio o el fin de una trama. Sin embargo, si esta secuencia aparece dentro del payload o carga útil, se introduce un bit de relleno para evitar confusiones.

En la figura, se puede ver el ejemplo para una secuencia de 6 bits de valor 1 (111111). En la carga útil, aparecen seis bits consecutivos con valor 1. Para evitar la confusión con la marca, se procesa el payload y se inserta un bit con valor 0 justo después del quinto bit. Al recibir los datos, si se detecta un 0 inmediatamente después de cinco bits de 1 dentro del payload, este se elimina para restaurar la secuencia original.

Este método es utilizado en protocolos High-level Data Link Control (HDLC). Este es un protocolo de enlace comúnmente empleado en redes WAN (Wide Area Network).

Marca y relleno de bits



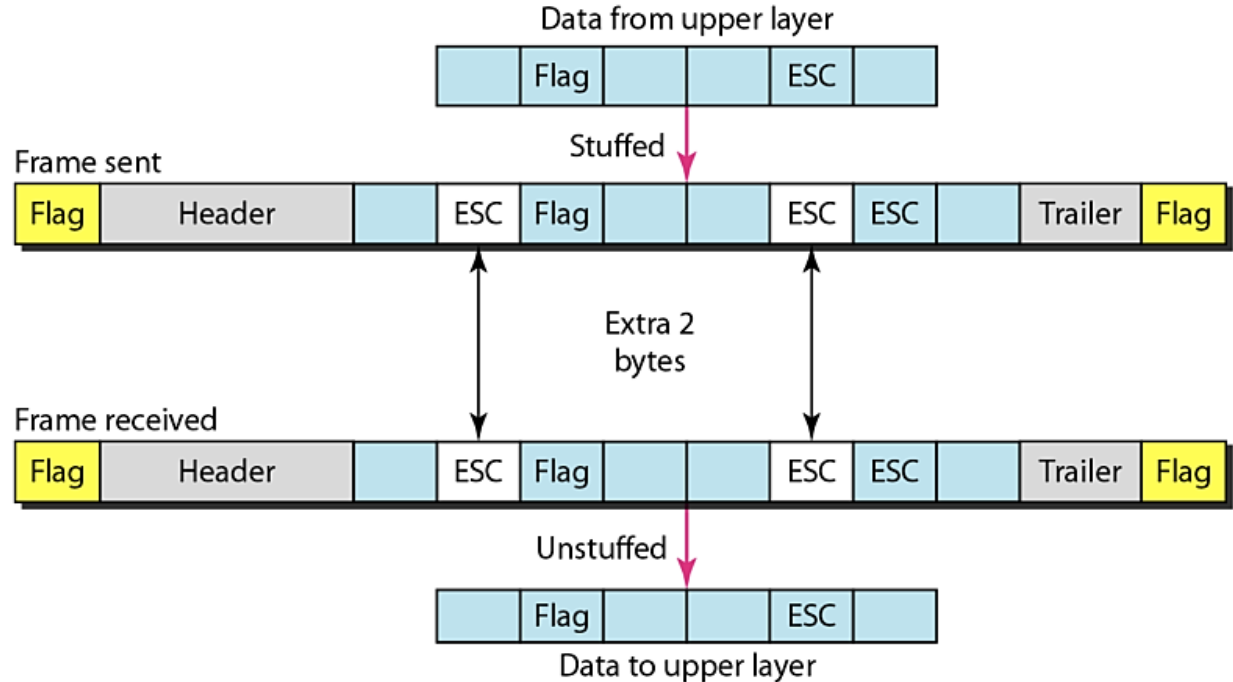
El método de marca y relleno de bytes es una mejora en el proceso de entramado. En este enfoque, se utiliza un byte especial, conocido como “flag”, para indicar el inicio y el final de cada trama. No obstante, surge un problema: el byte “flag” puede aparecer dentro de los datos que se están transmitiendo, lo que podría generar confusiones.

La solución a este problema es utilizar un byte de escape o “ESC”. Cada vez que el byte “flag” aparece en los datos, se añade este byte de escape antes del “flag” para que el receptor lo interprete correctamente. Sin embargo, esto presenta otro desafío, ya que el propio byte de escape de escape también podría aparecer en los datos. En ese caso, se resuelve utilizando otro byte de escape para señalar el uso del byte de escape original.

En resumen, las reglas básicas de este método son: (1) reemplazar cada “flag” en los datos con “ESC flag”, (2) reemplazar cada byte de escape dentro de los datos con “ESC ESC”. Esto asegura que los delimitadores no se confundan con los datos reales.

Este método se emplea en protocolos como PPP (Point to Point Protocol), que es un protocolo de enlace usado en el acceso a Internet a través de módems.

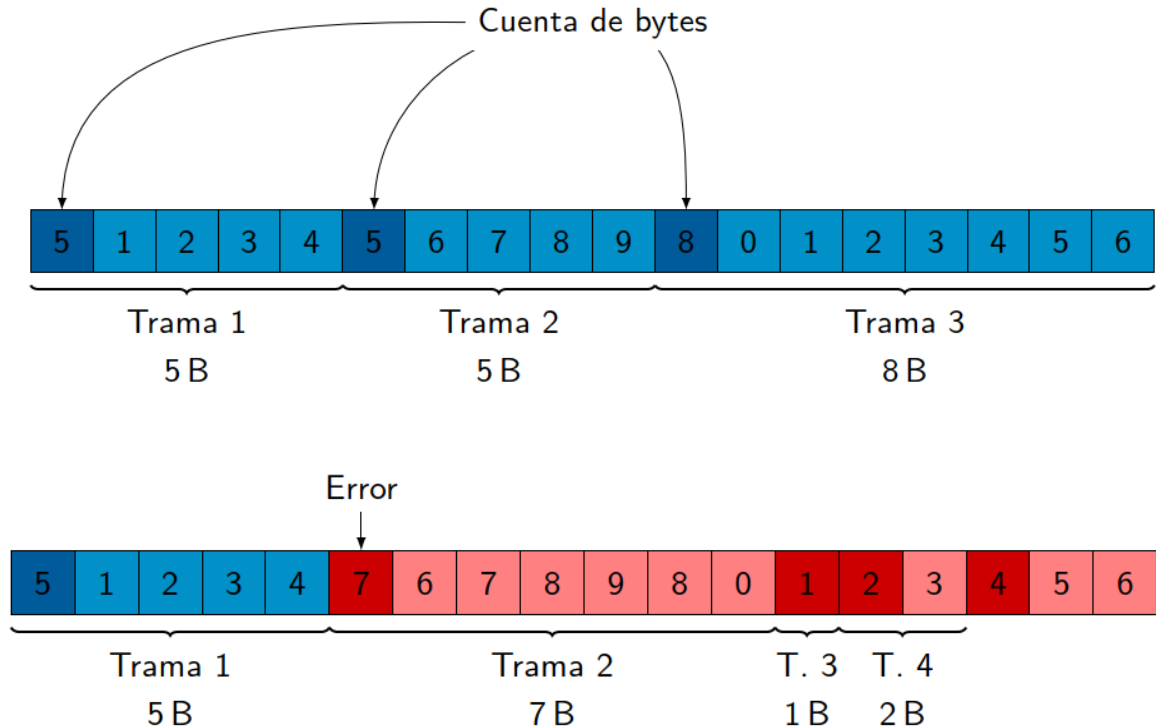
Marca y relleno de bytes



El método de cuenta de bytes es una técnica utilizada en el entramado para delimitar las tramas. La idea principal consiste en añadir al principio de cada trama un número que indica su tamaño en bytes. De esta manera, el receptor sabe cuántos bytes debe leer para completar la trama.

Sin embargo, este método tiene una desventaja importante. Si ocurre un error durante la transmisión, resulta muy difícil resincronizar el proceso. El caso más catastrófico sería un error en el tamaño de los bytes: no solo afecta a la trama donde se produjo, sino que también puede desincronizar las tramas siguientes. Esto complica todavía más la recuperación de la información.

Cuenta de bytes



Tratamiento de errores

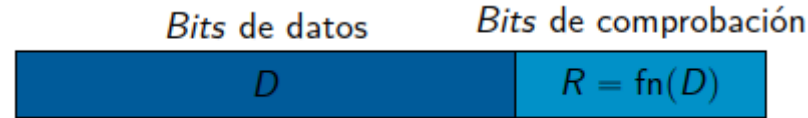
Códigos de Error

Algunos bits pueden alterarse al transmitirse a través de una red. Se cambian 0 por 1 y viceversa. Estos errores pueden ser causados por ruido térmico, interferencias electromagnéticas, etc. Pueden afectar a bits sueltos o a ráfagas de bits consecutivos. Hay dos opciones para tratar los errores. (1) Detectar los errores y retransmitir las tramas con errores. (2) corregir los errores.

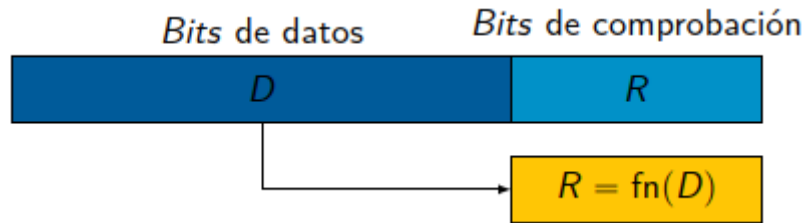
La detección es saber si hay bits erróneos. La corrección es ubicar los bits erróneos. La detección es obligatoria. La corrección es opcional, puesto que puede ser más eficiente reenviar el mensaje.

El tratamiento de errores requiere información extra: redundancia. La redundancia se usa en el alfabeto radiofónico donde cada letra se asocia a una palabra. Esto minimiza los errores, aunque exista mucho ruido en el canal.

Se asume que el nivel físico de la comunicación funciona con una calidad razonable. Por ejemplo, Ethernet tiene un índice de errores de bits (Bit Error Ratio, BER) menor a 10 ppb.



código bloque enviado emisor



cálculo redundancia receptor

Actualmente, se manejan dos tipos de códigos de corrección de errores: códigos bloque y códigos flujo. Los códigos bloque operan sobre bloques de bits de tamaño fijo, mientras que los códigos flujo operan sobre un flujo de bits.

En los códigos bloque, cada palabra código representa un bloque de datos y, por tanto, una cantidad determinada de bits de comprobación (o redundancia).

El transmisor calcula la redundancia R como una función de los bit de datos D de la carga útil y envía un mensaje de tamaño $N=D+R$. Por otro lado, el receptor recibe el mensaje con errores desconocidos y recalcula los bits de redundancia a partir de los bits de datos. Si el resultado no coincide con el recibido, hay errores en el mensaje.

La eficacia de un código de redundancia puede medirse de dos maneras. Una es mediante el overhead y la otra es mediante la tasa. Ambos miden el tamaño de la redundancia en comparación a la carga útil transmitida. Se busca que sea el menor overhead y la mayor tasa posible.

Tratamiento de Errores

COMPARISON OF EDAC CODES.

Error Correcting Code	Bit Overhead	Code Rate	Complexity		Error Correction Capability
			Time	Space	
Hamming(12,8,3)	50.00%	66.67%	$O(mn)$	$O(mn)$	SEC
Parity	6.25%	94.12%	$O(mn)$	$O(mn)$	SED
Rectangular parity	12.50%	91.43%	$O(mn)$	$O(mn)$	SEC
4-D parity	18.75%	84.21%	$O(mn)$	$O(mn)$	DEC
Golay(23,12,7)	91.67%	52.17%	$O(mk)$	$O(2^k)$	TEC
BCH(31,16,7)	93.75%	51.61%	$O(k^2)$	$O(mn)$	TEC

$$\frac{|R|}{|D|}$$

$$\frac{|R|}{|R| + |D|}$$

EDAC = Error Detection and Correction
 SEC = Single Error Correction
 SED = Single Error Detection
 DEC = Double Error Correction
 TEC = Triple Error Correction

Ejemplo

Supongamos un código sencillo para manejar errores: enviar dos copias del mensaje. Si hay bits que no coinciden, habrá errores. Por ejemplo, se envía el mensaje 0010|0010 y se recibe 0010|0110. ¡Hay errores!

¿Cuántos errores puede corregir? Cero. Observando el mensaje recibido, no se puede saber si el cambio se produjo en el mensaje original o la copia.

¿Cuántos errores puede detectar? Uno solamente. Si se produjesen un error en el mensaje y su duplicado, el receptor lo daría por válido. No saltaría la alarma.

¿Cuántos errores hacen falta para que falle? Dos. No puede detectar que un mismo bit haya cambiado dos veces.

D	R	D R
00	00	00 00
01	01	01 01
10	10	10 10
11	11	11 11

datos

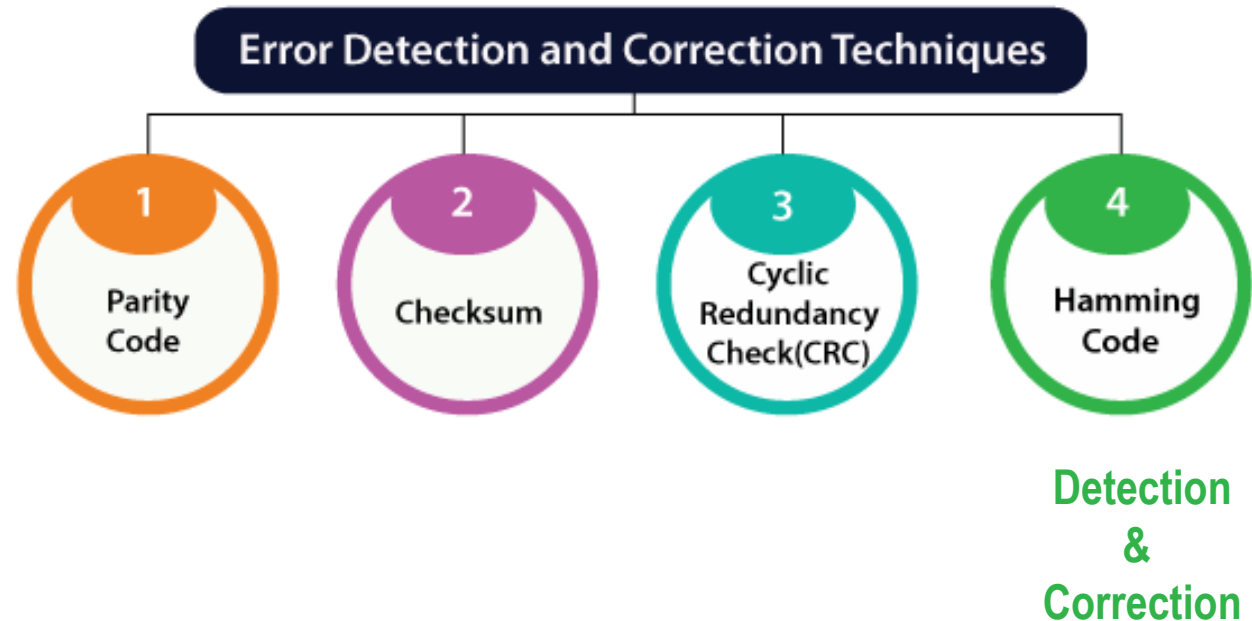
redundancia

código bloque

Detección de errores

A continuación, se verán varias técnicas de codificación que se utilizan en la práctica para detectar errores en la transmisión de datos. Estas técnicas incluyen el uso de bits de paridad, checksums y códigos de redundancia cíclica (CRC). La detección de errores permite arreglar el error mediante la retransmisión del mensaje.

Introducción



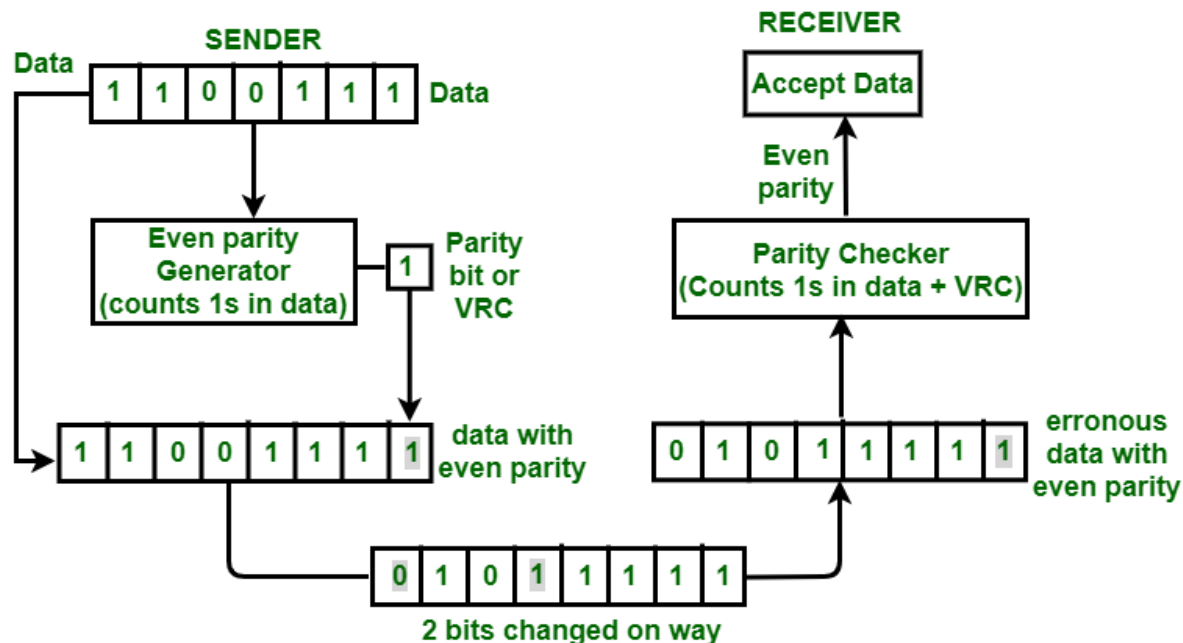
Bit de paridad

El bit de paridad es una técnica simple para detectar errores. Consiste en agregar 1 bit adicional a los datos transmitidos. Existen dos variantes del método de bit de paridad: paridad par y paridad impar.

(a) En la paridad par, se cuenta el número de bits que tienen valor 1. Si el total es impar, se establece el bit de paridad en 1, lo que hace que la suma total sea par. Si el conteo de bits 1 es par, se deja el bit de paridad en 0, ya que la suma total ya es par.

(b) En la paridad impar, se hace lo contrario. Se suman los bits que tienen valor 1, y si el resultado es impar, se establece el bit de paridad en 0. Si la suma de bits 1 es par, se establece el bit de paridad en 1, lo que hace que la suma final sea impar.

El bit de paridad puede detectar 1 error o un número de errores impar. Es incapaz de detectar un número par de errores. Tampoco tiene capacidad de corrección.



Checksum

El checksum se basa en la idea de sumar los datos en palabras de S bit. El transmisor calcula la suma y agrega su complementario al final del mensaje. El receptor calcula la suma del mensaje recibido: si el resultado final es cero, entonces no existe error de transmisión.

Cálculo del checksum por el emisor:

1. Se organizan los datos en palabras de S bit (en este caso, 8 bit).
2. Se agrega una palabra adicional con valor 0 (checksum) al final de los datos.
3. Se realiza la suma de todas las palabras. Si hubiera acarreo en la suma parcial, se añadiría. El acarreo es la parte de la suma que excede los S bit.
5. Se niega la suma final. El resultado es el checksum.

Verificación del checksum por el receptor:

1. Se organizan los datos en palabras de S bit. Una de ellas es el checksum calculado y enviado por el transmisor.
2. Se realiza la suma de todas las palabras, incluyendo la del checksum. Se suma cualquier acarreo parcial.
4. Se niega el resultado final de la suma. Se comprueba que valga 0. Esto implica que no ha habido errores.

Puede detectar 1 o ráfagas de error de varios bits que afecten a un bloque de S bits. No puede corregir errores.

Original Data

10011001	11100010	00100100	10000100	Checksum
----------	----------	----------	----------	----------

1 2 3 4
k=4, m=8

Sender

1	10011001
2	11100010
	101111011
	1
	01111100
3	00100100
	10100000
4	10000100
	100100100
	1
	00100101
Sum:	00100101
Checksum:	11011010

Receiver

1	10011001
2	11100010
	101111011
	1
	01111100
3	00100100
	10100000
4	10000100
	100100100
	1
	00100101
Checksum:	11011010
Sum:	11111111
Complement:	00000000
Conclusion:	Accept Data

CRC (Cyclic Redundancy Check) es un método de detección de errores que se basa en matemáticas discretas, específicamente en campos de Galois. Fue nombrado en honor a Evariste Galois, un matemático francés del siglo XIX.

Los mensajes se representan como polinomios. Por ejemplo, el mensaje 10011010 se puede representar como $x^7 + x^4 + x^3 + x^1$.

Para cada D bits de datos, se genera R bits de comprobación de manera que los $N = D + R$ bit. El tamaño de la redundancia es G-1 donde G es el tamaño del polinomio generador.

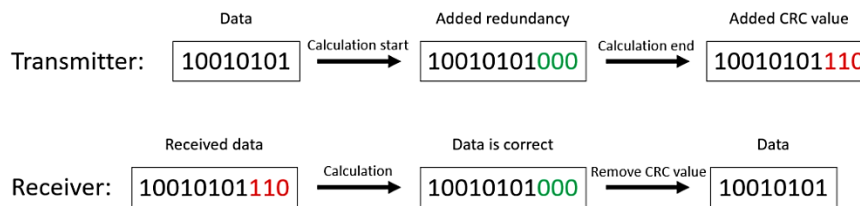
Transmisión:

1. Se extienden los **D bits** de datos con la redundancia. Esto es, se añaden (G-1) bit valor 0.
2. Se divide el polinomio del mensaje por el **polinomio generador G(x)** utilizando una operación lógica XOR. Emisor y receptor tienen una copia de este polinomio.
3. Se almacena el resto y se ignora el cociente.
4. Se introduce este **resto** en los R bits añadidos al principio.

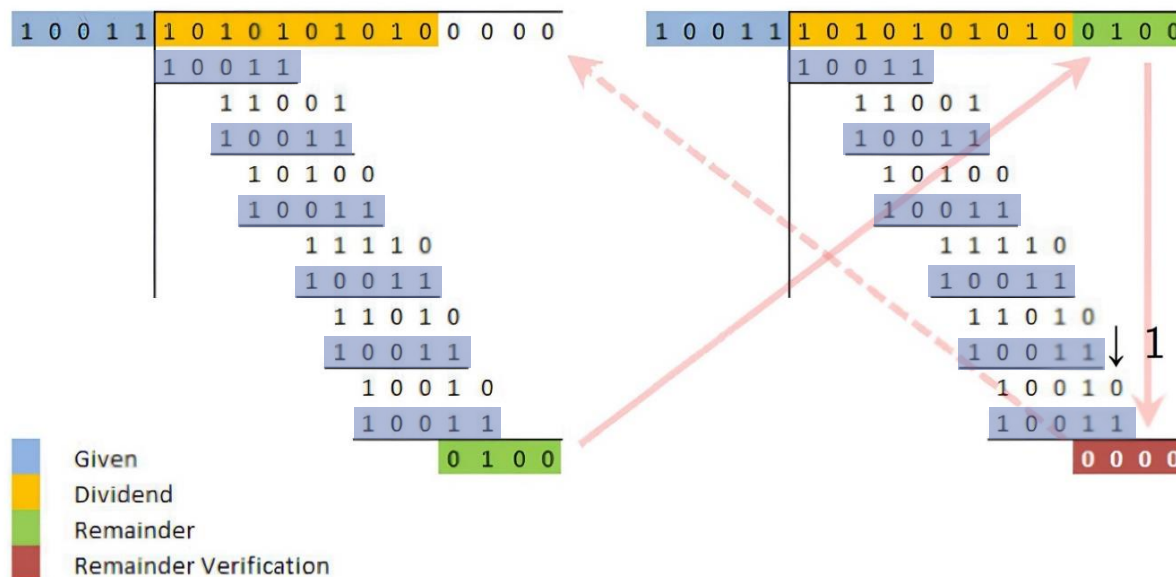
Recepción:

1. Se divide el mensaje por el **polinomio generador G(x)**.
2. Se comprueba si el **resto de verificación** es igual a 0. Si es así, no hay error. Si no, hay error.

CRC 1/2



Fuente: Cyclic Redundancy Check <https://nerd-corner.com/cyclic-redundancy-check-in-c-code/>



CRC 2/2

Para el estándar CRC32 usado en Ethernet, el polinomio generador es $G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$.

Es capaz de detectar 3 errores o un número impar de errores de bit. También es capaz de detectar ráfagas de errores de hasta $R = 32$ bits.

Dependiendo del tamaño del CRC, es capaz de detectar ráfagas de errores, lo que incrementa la eficiencia de la comunicación. No obstante, el uso de CRC presenta una desventaja importante: aunque puede detectar errores de múltiples bits, no tiene la capacidad de corregirlos. En su lugar, descarta la información y solicita al usuario que vuelva a enviar los datos. Por esta razón, en casos simples como un error de un solo bit, implementar CRC sería más lento en comparación con un Código de Hamming. Además, los Códigos de Hamming pueden detectar un mayor porcentaje de errores por longitud de código en comparación con CRC a medida que la longitud del código aumenta.

[illegible]

Binary form: 110101100000000000000000000000000000

Divided by: 10001110110111000110111101000001

Quotient / Result is :11011111

Remainder / CRC bits: 10011010000111001111001000011111

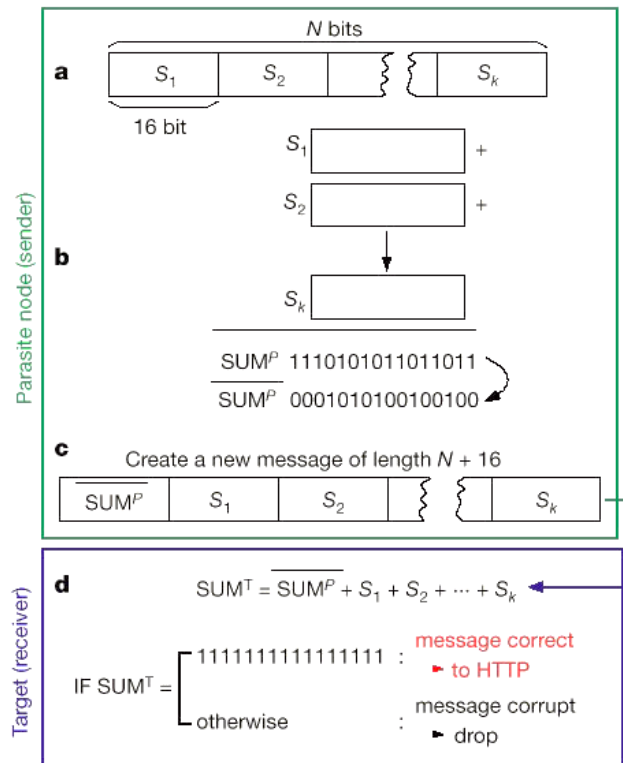
Data 1101011010011010000111001111001000011111 is sent

<https://stackoverflow.com/questions/76096237/how-to-calculate-crc32-with-the-rules-of-polynomial-division-xor-division-into>

Los **códigos CRC** (Cyclic Redundancy Check) se utilizan ampliamente en la capa de enlace de comunicación como Ethernet, WiFi, ADSL, etc. La protección depende del polinomio generador. Los **checksums** se utilizan en los protocolos de las capas altas de Internet, como IP (red), TCP y UDP (transporte).

La **paridad** apenas se utiliza en las redes, ya que su protección es pobre en ese entorno. Sin embargo, la paridad tiene otros usos, como en sistemas de almacenamiento RAID (Redundant Array of Independent Disks) o en buses PCI (Peripheral Component Interconnect).

Detección de errores en la práctica



TCP checksum

Corrección de errores

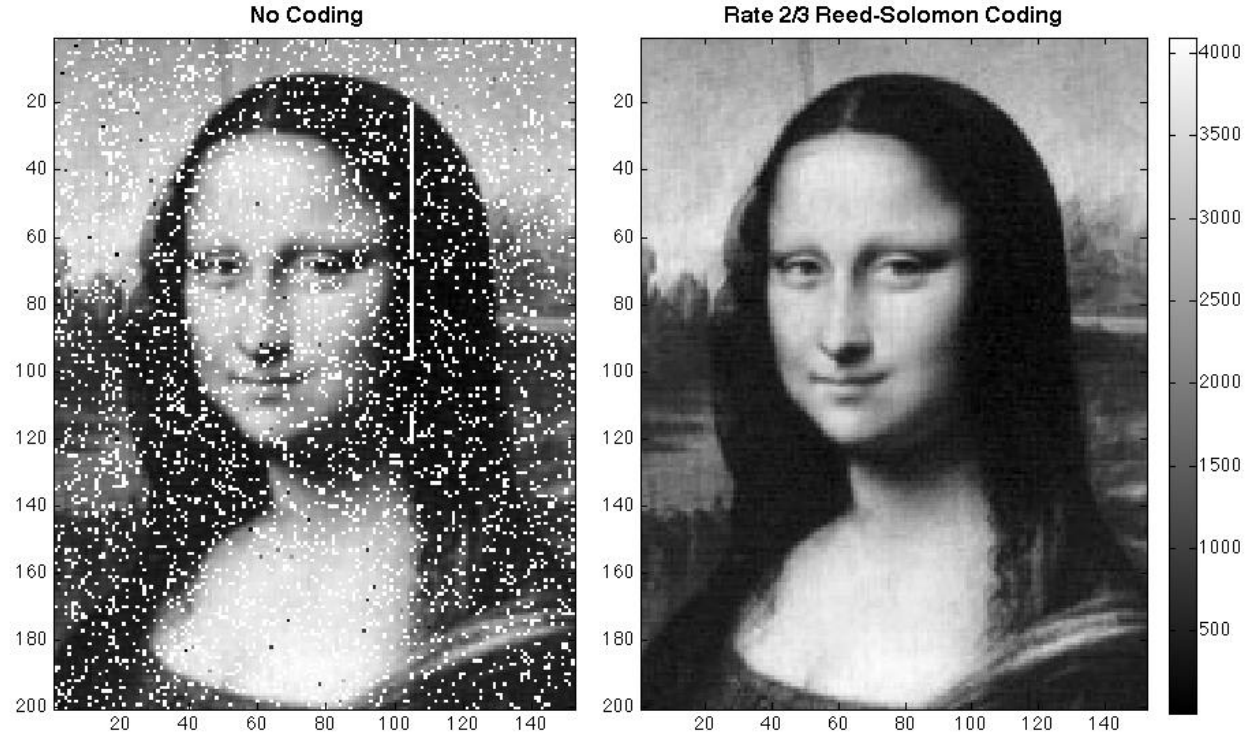
Introducción

Existen varios códigos que se usan en la práctica para corregir errores: Hamming, convolucionales, Reed-Solomon, Low-Density Parity-Check (LDPC) y Turbo códigos.

Los códigos de corrección de errores se utilizan de forma generalizada en la capa física. Hasta hace unos años, se utilizaban principalmente códigos convolucionales. En nuevos sistemas, se están utilizando alternativas mejores como LDPC o Turbo códigos.

Además de la capa física, la corrección de errores también se emplea en la capa de aplicación, especialmente en aplicaciones de multimedia interactivo que requieren tiempo real.

En las capas intermedias (enlace, red y transporte), se suele utilizar la retransmisión cuando se detecta un error. Es decir, en caso de detectarse un error, se vuelve a transmitir el mensaje íntegramente. No se usa la corrección de errores.



¿Por qué no se puede usar siempre corrección de errores? Si los bits fiables se utilizaran para acotar la posición del error, la corrección sería sencilla. Sin embargo, puede haber errores tanto en los bits de comprobación como en los de datos. Incluso los datos podrían no contener errores y haberlos solo en la redundancia.

En general, para corregir errores es necesario añadir más redundancia en comparación con la cantidad requerida para simplemente detectarlos. Esto se debe a que la corrección de errores demanda una mayor distancia de Hamming que para la detección. Como resultado, corregir errores implica un mayor coste, tanto en términos de la tasa efectiva de transmisión de datos como en el tiempo de procesamiento. La elección entre usar códigos detectores o correctores dependerá del escenario.

Se debe usar corrección de errores cuando: se esperan muchos errores (canal muy ruidoso), las retransmisiones son difíciles (por ejemplo, en enlaces muy largos o interplanetarios) o no hay tiempo para retransmisiones (aplicaciones en tiempo real).

Detección vs corrección

Single bit error: In a frame, there is only one bit, anywhere though, which is corrupt.



Multi bits error: Frame is received with more than one bits in corrupted state.



Burst bits error: Frame contains more than 1 consecutive bits corrupted..



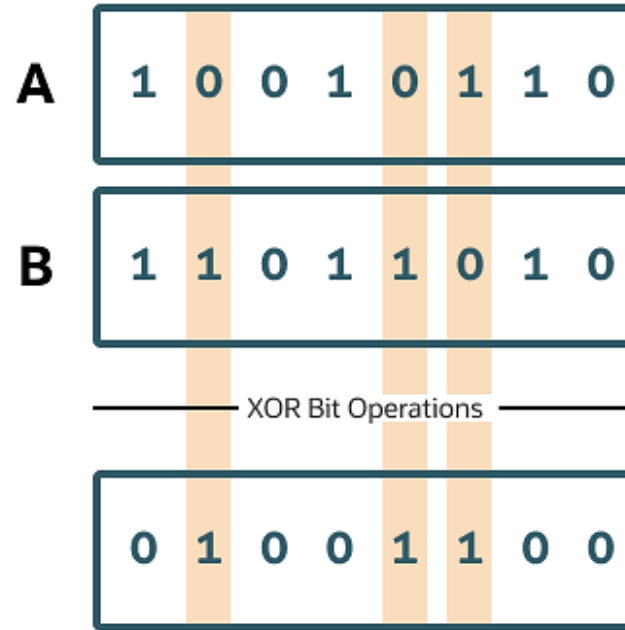
R. W. Hamming (1915-1998) fue un pionero en el campo de los códigos de detección y corrección de errores. En 1950, publicó el libro "Error Detecting and Error Correcting Codes".

La **distancia Hamming entre dos vectores de bits** es el número de bits que se necesitan cambiar para convertir una secuencia de bits en otra. Por ejemplo, la distancia Hamming entre las secuencias 1010 y 1100 es de 2.

La distancia de un código es la distancia Hamming mínima entre dos palabras cualesquiera de ese código. Por ejemplo, sea un código que manda bits por triplicado: $D1|R1=0|00$, $D2|R2=1|11$. La distancia Hamming $DH(\{000,111\}) = 3$.

En los razonamientos previos, se usa el mensaje al completo. Se llama mensaje a la información enviada. Está formado por la carga útil (payload), cabeceras y la redundancia.

Distancia Hamming 1/2



Hamming Distance = 3

La **distancia Hamming de un código** se utiliza para detectar y corregir errores. Si un código tiene una distancia Hamming DH, se pueden detectar hasta $d = DH-1$ errores. En el código que manda bits por triplicado, $d = DH-1 = 2$. Se pueden detectar uno o dos errores (p.e. 001, 010, 011, 100 o 110).

Para un código de distancia Hamming DH, se pueden corregir $d = (DH-1)/2$ errores. Si se recibe un mensaje con errores, se puede corregir mediante la asignación al código con menor distancia. Esta es la regla de máxima verosimilitud. Por ejemplo, en un código que manda bits por triplicado $DH = 3$, se puede corregir un error. Si se recibe 001, 010, 011, 100 o 110, se detecta el error y decide $D = 0$. Si se recibe 011, 101 o 110, se detecta el error y se decide $D=1$.

Distancia Hamming 2/2

$w_1:$	1	1	0	1	0	0	0
$w_2:$	0	1	1	0	1	0	0
$w_3:$	1	1	1	0	0	1	0
$w_4:$	1	0	1	0	0	0	1

Distancia Hamming (DH) = 3

detecta: $DH-1 = 2$ err

corrige: $(DH-1)/2 = 1$ err

Método para construir códigos de bloque con distancia de Hamming (DH) igual a 3. Al ser DH=3, pueden detectar hasta 2 errores y corregir 1 error.

Se trata de códigos del tipo:

Hamming(N,D):

- Longitud de la palabra código: $N = 2^R - 1$
- Longitud de los datos: $D = 2^R - r - 1$
- En ambos casos, $R \geq 2$

Ejemplos:

- $R = 3 \rightarrow \text{Hamming}(7,4)$
- $R = 7 \rightarrow \text{Hamming}(127,120)$.

Los códigos Hamming son ejemplos de códigos lineales: se pueden codificar/verificar mediante matrices: la matriz generadora (G) para codificar y la matriz de comprobación de paridad (H) para verificar.

Códigos Hamming

matriz generadora Hamming(7,4)

$$G_{D \times N} = \begin{pmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 \end{pmatrix}_{4 \times 7}$$

matriz comprobadora paridad Hamming(7,4)

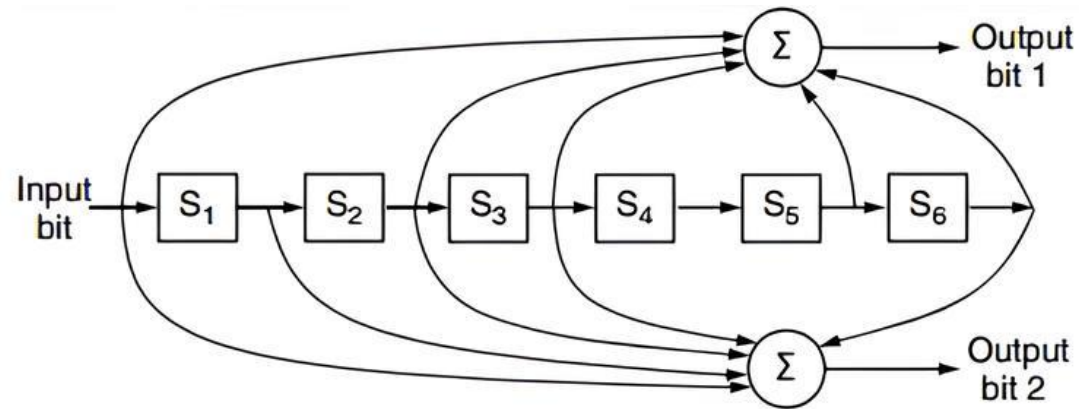
$$H_{R \times N} = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{pmatrix}_{3 \times 7}$$

Los códigos convolucionales son una familia de códigos de corrección de errores que operan sobre flujos de bits en lugar de bloques de bits. Estos códigos son conocidos como códigos flujo porque cada bit de salida depende del bit de entrada actual y de otros anteriores.

Los códigos convolucionales son códigos con memoria. Esto permite la consulta de valores de bits anteriores. El algoritmo de Viterbi es comúnmente utilizado para decodificar estos códigos: dada una secuencia de eventos observados, encontrar la secuencia de estados más probable que los ha generado.

Estos códigos son utilizados por la NASA en sus misiones Voyager. En ellas, se realizan transmisiones a las sondas Voyager. En junio de 2021, la Voyager 1 se encontraba a una distancia de 153,14 UA (22 909 417 919 km) del Sol, siendo la nave espacial más alejada de la Tierra.

Códigos Convolucionales



Código convolucional binario usado por la NASA en 802.11

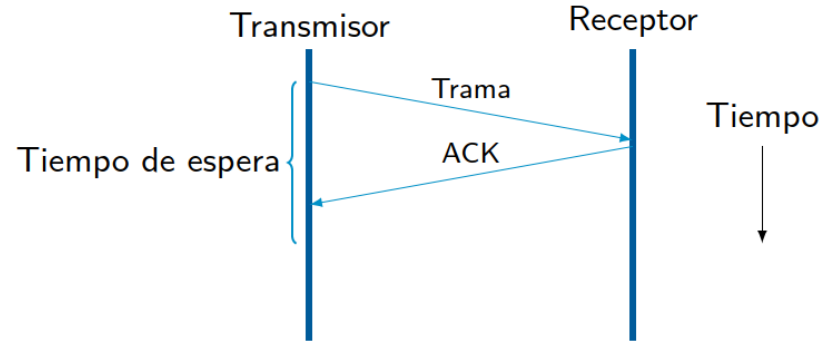
Retransmisiones

Cuando se produce un error en la transmisión de datos, existen dos estrategias para manejarlo: corregir el error con un código de corrección de errores o detectar el error y retransmitir la trama. La segunda estrategia se conoce como ARQ (Automatic Repeat reQuest). Se utiliza en entornos donde los errores son frecuentes y es importante su recuperación.

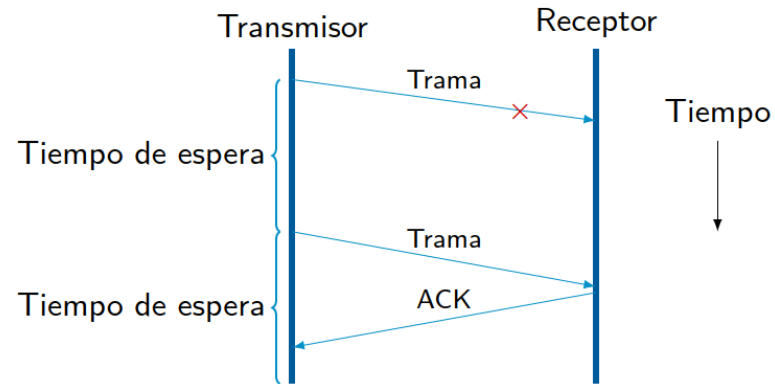
El ARQ se utiliza en diferentes niveles del modelo TCP. (1) capa de enlace (WiFi), para garantizar la fiabilidad en un enlace específico, (2) capa de transporte para garantizar la fiabilidad extremo a extremo y (3) capa de aplicación para garantizar la fiabilidad extremo a extremo cuando no se utiliza un transporte fiable.

Para implementar el ARQ, se establecen reglas en el transmisor y el receptor. El receptor automáticamente asiente las tramas correctas con un ACK (acknowledgement), mientras que el transmisor automáticamente reenvía la trama después de un tiempo de espera hasta que se reciba su ACK. De esta manera, se garantiza la fiabilidad en la transmisión de datos y se evitan las pérdidas de información.

operación normal (sin pérdidas)



error y retransmisión



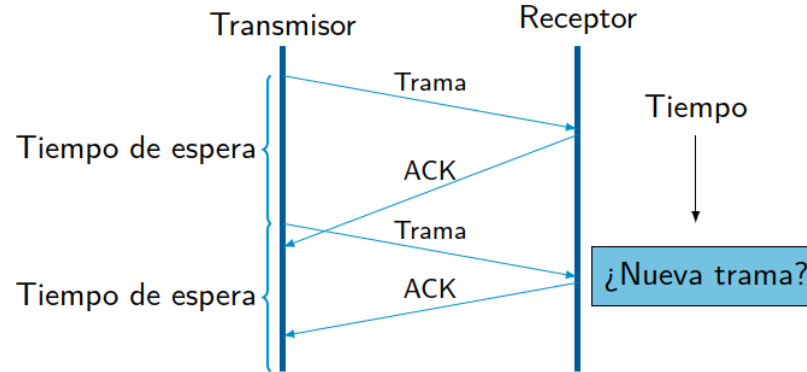
Aunque el protocolo ARQ (Automatic Repeat reQuest) es una solución efectiva para manejar errores en la transmisión de datos, existen dos cuestiones importantes que deben ser consideradas en su diseño.

La primera cuestión es determinar el **tiempo de espera** adecuado entre la transmisión de una trama y la retransmisión en caso de error. Un tiempo de espera demasiado grande puede dejar el enlace desocupado durante demasiado tiempo en caso de error, mientras que un tiempo de espera demasiado pequeño puede generar retransmisiones innecesarias. En una LAN, es relativamente fácil fijar un tiempo de espera adecuado, ya que se puede predecir el caso peor. Sin embargo, en Internet, la comunicación extremo a extremo atraviesa varios enlaces con características diferentes, lo que hace que sea muy difícil fijar un tiempo de espera adecuado.

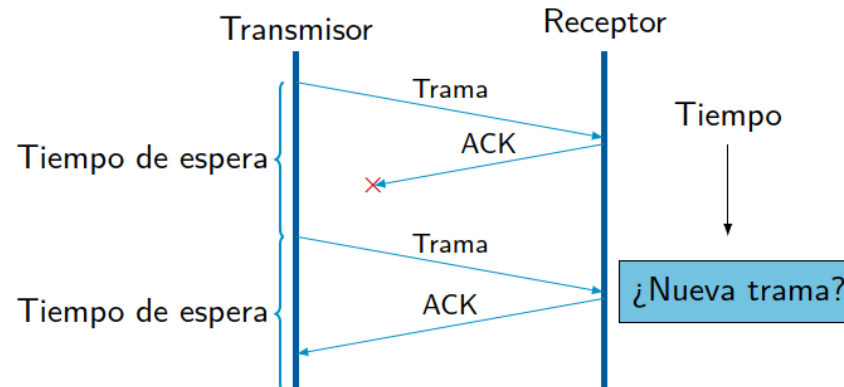
La segunda cuestión es cómo evitar aceptar **tramas duplicadas** por pérdida de ACK como nuevas tramas. Esto es crucial para garantizar que la información llegue correctamente al receptor.

Problemas ARQ

tiempo de espera demasiado corto



mensaje duplicado por pérdida de ACK



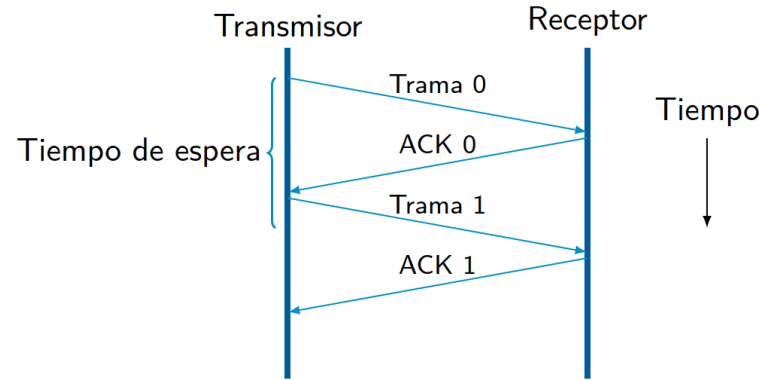
Para que el protocolo ARQ (Automatic Repeat reQuest) funcione correctamente, las tramas y los ACK (acknowledgements) deben contener números de secuencia. Esto es necesario para evitar la duplicación de tramas y para ordenar los paquetes recibidos.

En su forma más básica, el protocolo ARQ **utiliza un solo bit** para distinguir la trama actual de la siguiente. Esto se logra utilizando dos números: 0 y 1. Esta estrategia se conoce como "parada y espera". Aquí está el funcionamiento del protocolo: (1) el transmisor envía un paquete y se detiene, (2) espera a recibir el ACK de dicho paquete, (3) si el ACK llega, se envía el siguiente paquete, (4) si no llega, se retransmite el último paquete enviado.

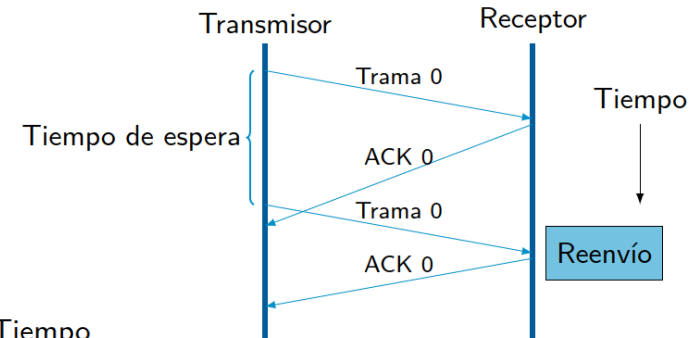
Este protocolo es simple, pero efectivo para garantizar la fiabilidad en la transmisión de datos.

Números de Secuencia

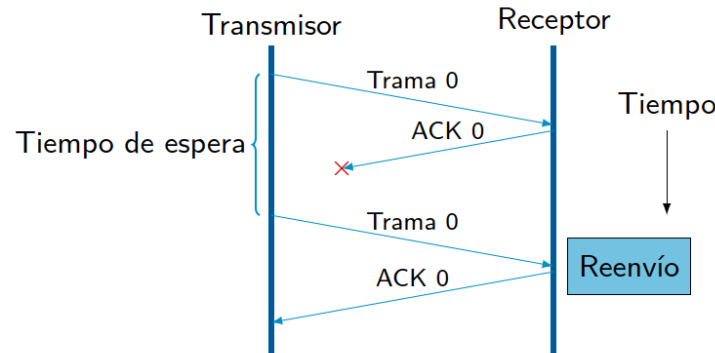
caso normal



tiempo espera demasiado corto



pérdida ACK

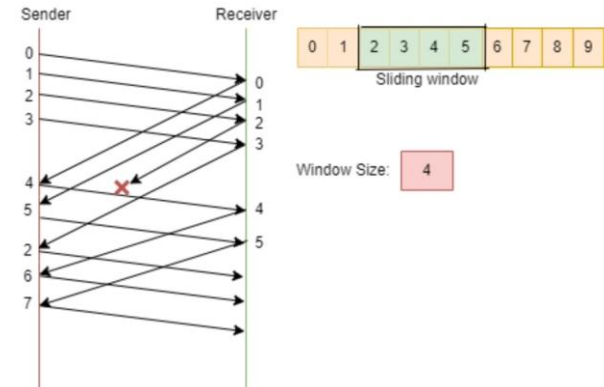
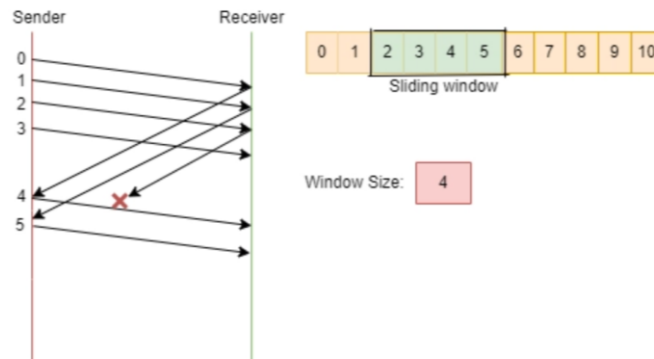
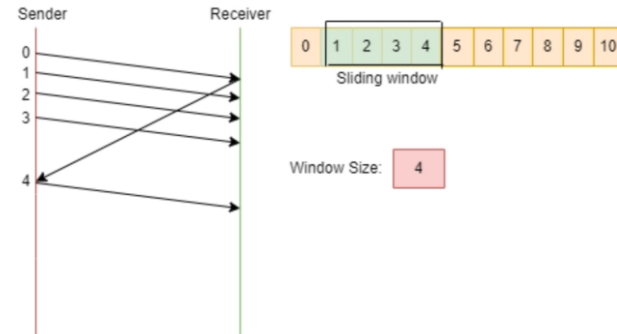
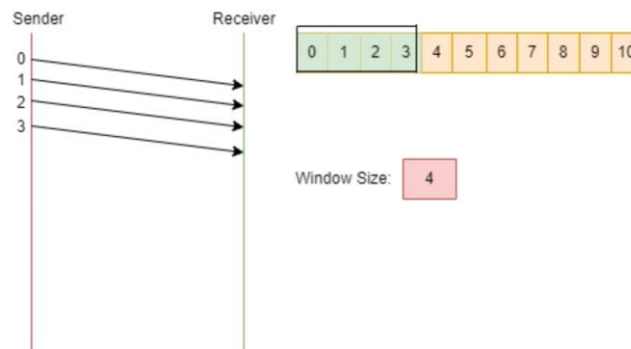


El protocolo de parada y espera (Stop and Wait) solo permite **una trama pendiente de asentimiento** a la vez. Esto significa que el emisor debe esperar a recibir un asentimiento (ACK) antes de enviar una nueva trama.

Supongamos un enlace con una tasa de transmisión R de 1 Mbit/s y un retardo D de 50 ms. ¿Cuántas tramas se pueden enviar por segundo? La respuesta es que, como mínimo, debemos esperar el tiempo de ida de la trama y vuelta del asentimiento, conocido como Round Trip Time (RTT). El RTT es como mínimo el doble de la latencia. En este caso, el RTT es de 100 ms. Por lo tanto, en 1 segundo, se pueden enviar como máximo 10 tramas.

Es importante destacar que este resultado no depende del ancho de banda del canal R , sino de la latencia del enlace. Esto hace que este protocolo sea aceptable para una LAN, pero no sea eficiente cuando el retardo es alto, como en la comunicación extremo a extremo a nivel de transporte. En ese caso, se necesitan técnicas más eficientes, como la ventana deslizante, que se abordarán en la capa de transporte.

Limitaciones ARQ



Fuente: Go-Back-N ARQ Protocol <https://www.naukri.com/code360/library/go-back-n-arq-protocol/>

Control de acceso al medio

En una red, es común que varios nodos quieran utilizar un enlace al mismo tiempo. Sin embargo, si todos "hablan" a la vez por el mismo canal físico, se producirán errores. Para evitar esto, la capa de enlace utiliza técnicas de control de acceso al medio, también conocidas como CSMA (Carrier Sense Multiple Access).

Existen dos tipos de protocolos para resolver el problema de acceso múltiple a un medio compartido:

(1) Aleatorio. Los nodos aleatorizan sus intentos de acceso al medio. Este tipo de protocolo es adecuado para situaciones con baja carga.

(2) Libre de contención. Los nodos ordenan sus intentos de acceso al recurso. Este tipo de protocolo es adecuado para situaciones con alta carga o en las que hay requisitos de calidad de servicio.

En esta sección, se profundizará sobre dos mecanismos concretos de control de acceso al medio: (1) Aleatorio: CSMA/CD empleado en Ethernet clásico. (2) Libre de contención: CSMA/CA o MACA empleado en redes WiFi.

Acceso Múltiple

Factors	CSMA CD	CSMA CA
Collision Handling	Detects collisions after they occur and stops transmission.	Avoids collisions before they occur by sensing the channel.
Network Type	Used mostly in wired networks where collision detection is possible.	Used mostly in wireless networks where collision detection is not possible.
Efficiency	Reduces the recovery time after a collision and is more efficient than CSMA CA.	Minimizes the possibility of collision and is less efficient than CSMA CD.
Data Frame Transmission	Resends the data frame whenever a conflict occurs.	Sends an intent to transmit before data transmission.
IEEE Standard	Used in 802.3 standard (Ethernet)	Used in 802.11 standard (Wi-Fi)
Similarity to simple CSMA	More efficient than simple CSMA.	Similar to simple CSMA.

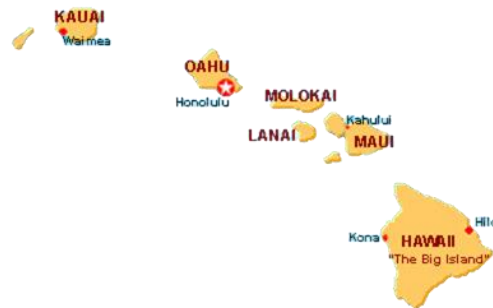
Fuente:CSMA CD vs CSMA CA – What's the difference? <https://www.pynetlabs.com/csma-cd-vs-csma-ca/>

En una red Ethernet, los nodos comparten un enlace, sin que ninguno de ellos esté a cargo de dar acceso. El protocolo de acceso múltiple es aleatorio. Se inspira en el protocolo ALOHA.

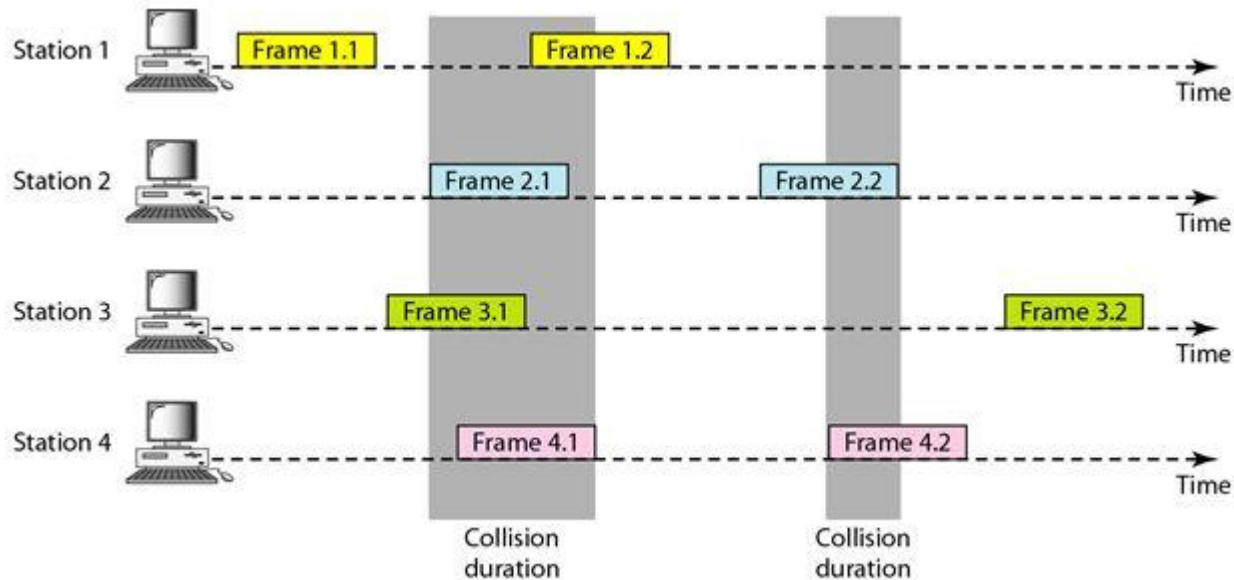
El protocolo ALOHA fue desarrollado en la década de 1960 para conectar por radio las islas hawaianas. La idea era muy sencilla: un nodo transmite en cuanto tiene tráfico, si no recibe un ACK después de un tiempo, espera otro tiempo aleatorio y retransmite. Aunque este protocolo era simple, tenía un problema: las tramas se perdían por colisión, alcanzando una eficiencia máxima del 18%.

El protocolo ALOHA inspiró a Bob Metcalfe a inventar Ethernet en 1973. Ethernet era una tecnología para redes locales (LAN) que permitía a los nodos compartir un cable coaxial a 10 Mbit/s. Ethernet se volvió muy popular en las décadas de 1980 y 1990.

El mecanismo de control de acceso al medio en Ethernet es CSMA/CD (Carrier Sense Multiple Access with Collision Detection). No es tan sencillo como el protocolo ALOHA, pero es más eficiente.



Fuente: The ALOHA Protocol <http://www.wirelesscommunication.nl/reference/chaptr06/aloha/aloha.htm>



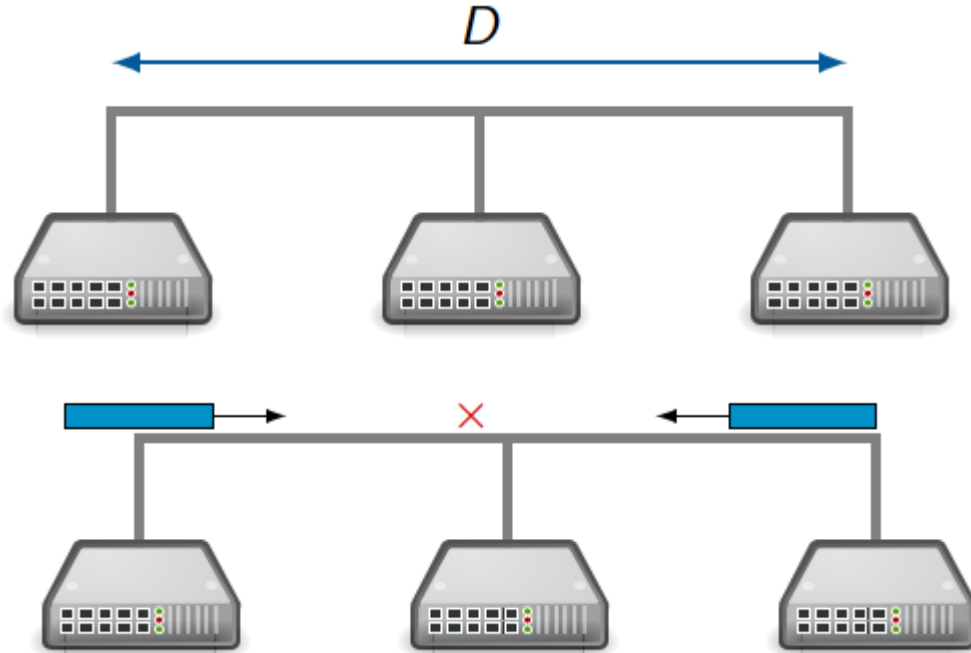
Fuente: Aloha Protocols <https://www.myreadingroom.co.in/notes-and-studymaterial/68-dcn/819-aloha-protocols.html>

El Acceso Múltiple con Portadora (Carrier Sense Multiple Access, CSMA) es un método de acceso múltiple que mejora el ALOHA al escuchar el canal antes de transmitir. Sin embargo, esto no es suficiente para eliminar las colisiones. Aunque un nodo puede escuchar el canal, es posible que no detecte la actividad de otro nodo que esté transmitiendo debido al retardo.

¿Cuánto tiempo tarda un nodo en enterarse de la colisión? En el peor caso, la colisión se produce entre dos transmisores en los extremos de un enlace con retardo D . En ese caso, el tiempo máximo necesario para detectar la colisión sería de $2D$: se consumiría D tiempo a la ida y D tiempo a la vuelta. Para garantizar que el nodo esté transmitiendo y escuchando el canal cuando llega el aviso de colisión, el nodo debe transmitir de forma ininterrumpida $2D$ unidades de tiempo. Debido a esta condición, el Ethernet clásico estableció el tamaño mínimo de trama en 64 bytes. Ese tamaño garantiza que el tiempo de transmisión sea mayor que la latencia de ida y vuelta.

Para la detección de colisiones (Collision Detection, CD), los nodos que transmiten escuchan el canal y si detectan niveles de señal más altos de lo transmitido, se considera una colisión. En ese caso, se detiene la transmisión y se envía un mensaje de jam para avisar al resto.

CSMA/CD 2/3



CSMA/CD 3/3

¿Qué debería hacer un nodo si otro está transmitiendo? Una idea es esperar a que termine y transmitir en cuanto acabe. Sin embargo, si hay varios nodos esperando, se podrían producir colisiones de intentos sucesivos.

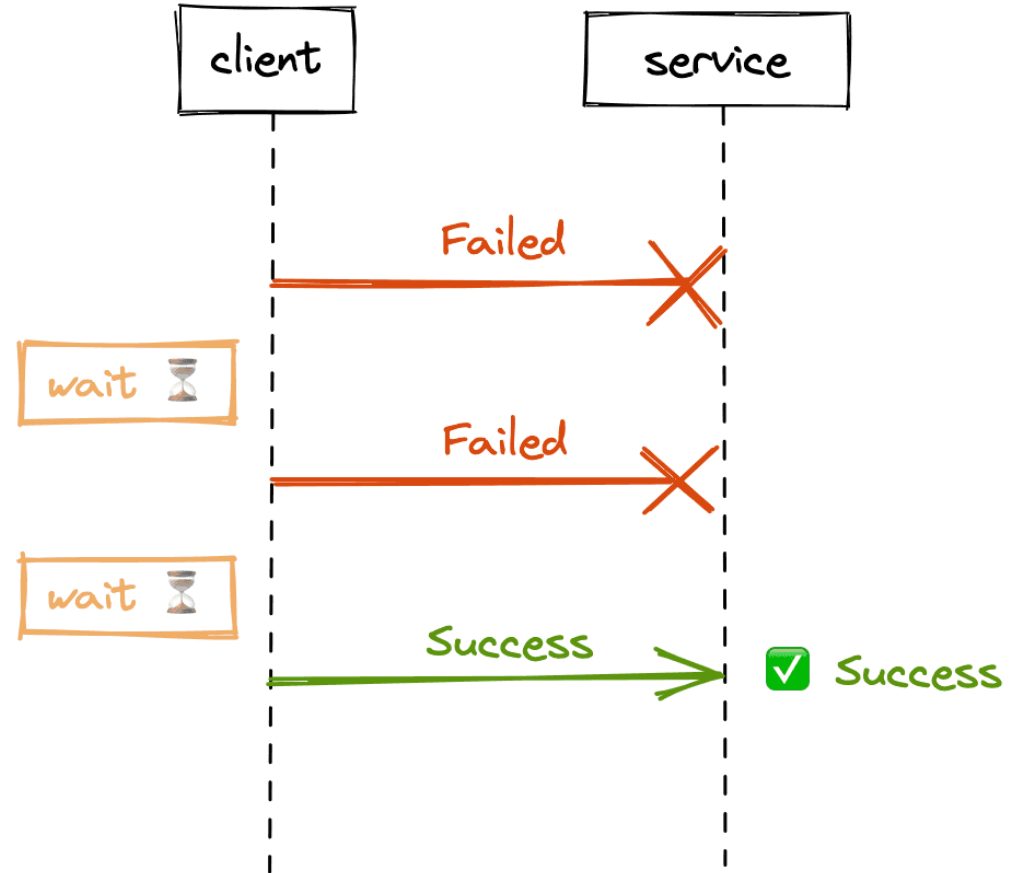
En la práctica, Ethernet clásico emplea un mecanismo llamado **retroceso exponencial binario** (Binary Exponential Backoff, BEB) para resolver este problema. Es una forma de estimar la probabilidad de colisión. Cuando el canal está libre y hay una trama que transmitir, se hace. Si se produce colisión, el comportamiento del protocolo se ajusta dinámicamente.

1.ª colisión: Espera entre 0 y 1 veces el tiempo de una trama y reintenta.

2.ª colisión: Espera entre 0 y 3 veces el tiempo de una trama y reintenta.

3.ª colisión: Espera entre 0 y 7 veces el tiempo de una trama y reintenta.

Y así sucesivamente. BEB dobla el intervalo en cada colisión sucesiva, lo que rápidamente hace que el intervalo sea suficientemente grande para funcionar. En la práctica, BEB es muy eficiente.



Las redes inalámbricas son mucho más complejas que las cableadas. Los nodos pueden tener áreas de cobertura diferente, dificultan la implementación de estrategias basadas en detección de colisiones.

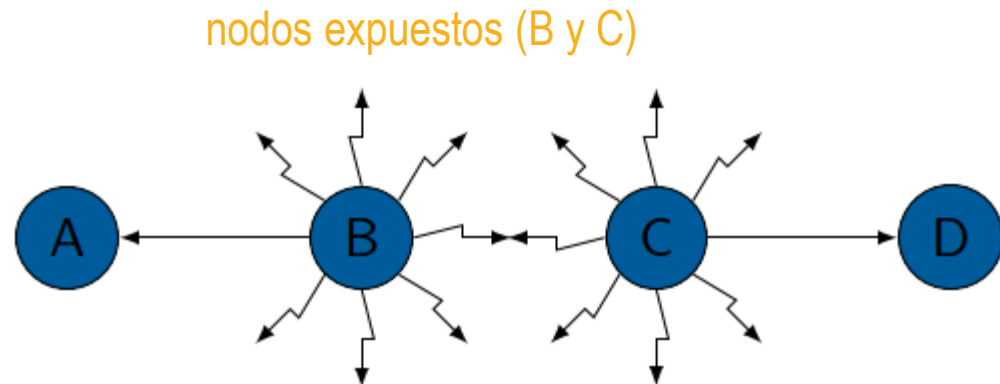
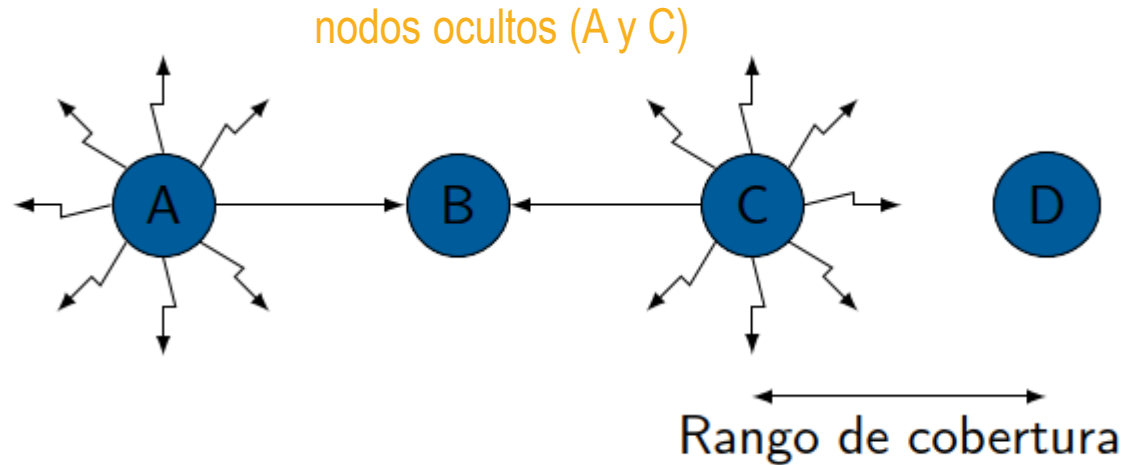
Además, recibir y transmitir al mismo tiempo en la misma banda es complejo, lo que dificulta la implementación de estrategias de detección de colisiones.

Las señales inalámbricas se reciben en las cercanías, siempre y cuando haya suficiente SNR (Signal-to-Noise Ratio).

Este define el rango de cobertura.

Dos nodos son **terminales ocultos** cuando transmiten a un tercero y no se pueden escuchar el uno al otro para evitar colisionar. Se necesita un mecanismo que evite la ineficiencia de las colisiones.

Dos nodos son **terminales expuestos** cuando transmiten y se pueden escuchar el uno al otro, aunque no se producen colisiones cuando transmiten a A y a D. Se necesita un mecanismo que permita transmitir concurrentemente cuando sea posible para incrementar la eficiencia.



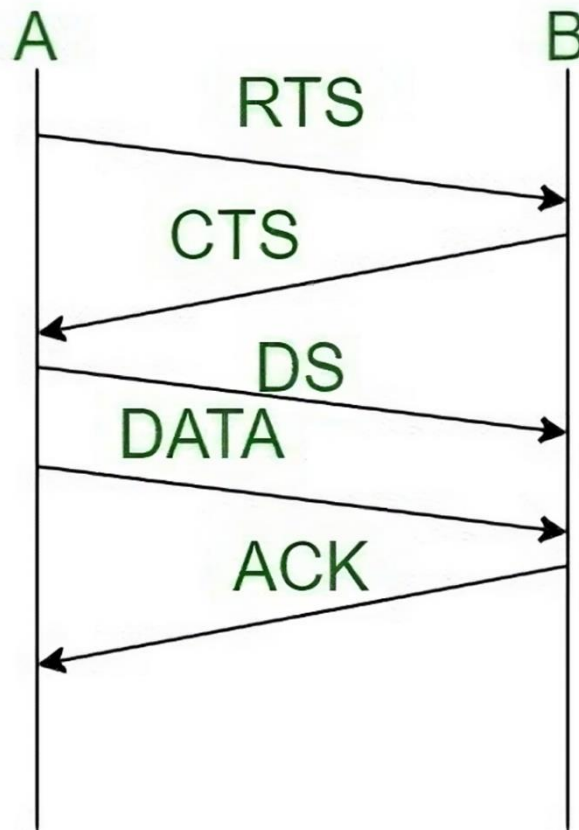
MACA (Multiple Access with Collision Avoidance) es un protocolo de acceso múltiple que utiliza un breve handshake (apretón de manos) en lugar de CSMA. Se "pide permiso" antes de transmitir.

El mecanismo utilizado en WiFi (802.11) está inspirado en MACA. MACAW (MACA for Wireless) es una versión más compleja de MACA. El protocolo MACAW funciona de la siguiente manera:

- (1) Un transmisor envía la trama RTS (Request-To-Send) con la longitud de la trama.
- (2) El receptor envía la trama CTS (Clear-To-Send) con la longitud de la trama.
- (3) El transmisor envía la trama DS (Data-Sending) proporcionando información sobre la longitud del campo de datos. El campo DS es opcional.
- (4) El transmisor envía los datos o DATA.
- (5) El receptor da gracias (ACK). Esto es una forma de finalizar la transmisión, avisando que ha sido correcta.

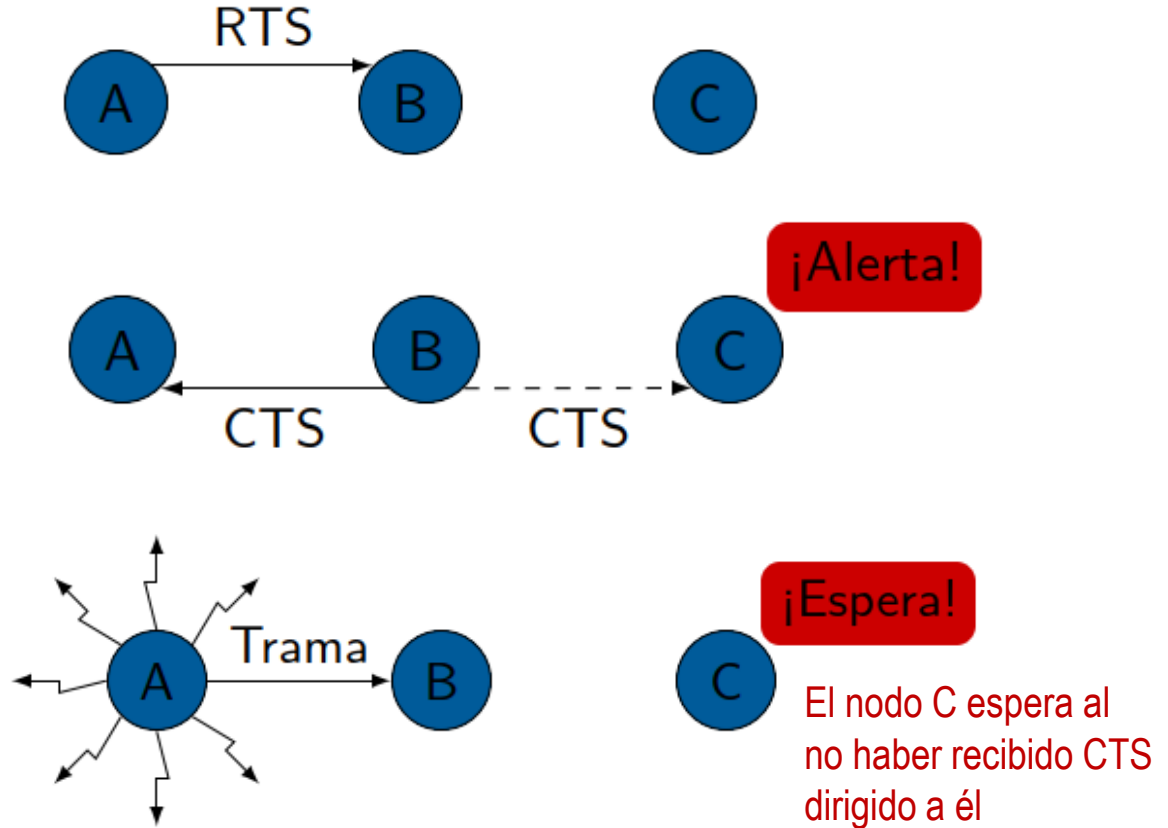
Las colisiones en los RTS y los CTS son todavía posibles, pero la probabilidad es muy baja debido a que son mensajes pequeños.

MACA 2/4



En el siguiente ejemplo, A transmite a B con C como **terminal oculto**:

- (1) A envía un RTS a B.
- (2) B envía un CTS a A y llega a C también.
- (3) A envía la trama mientras C espera.

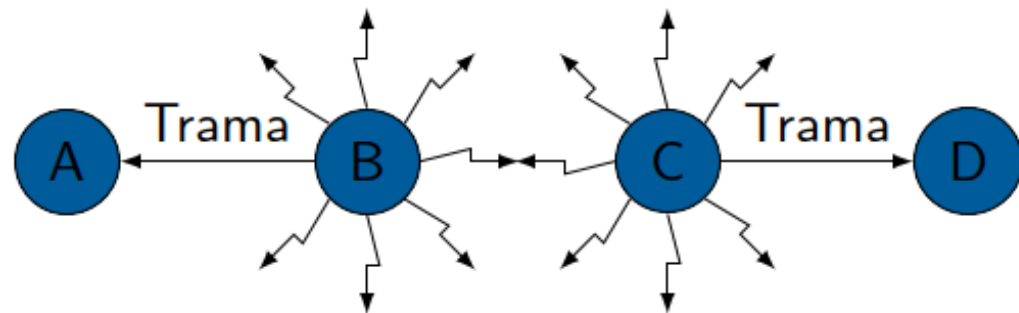
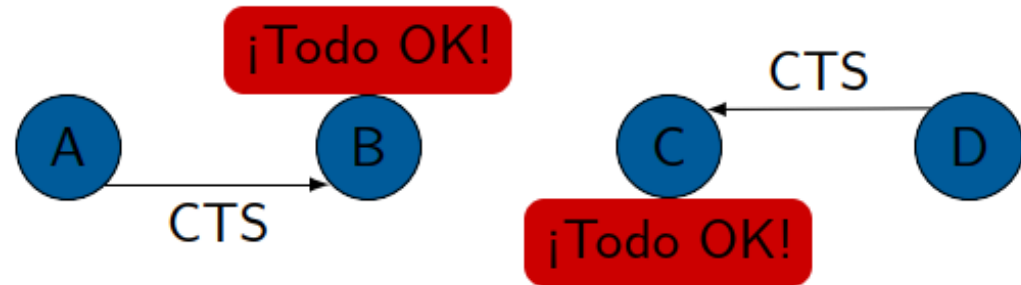
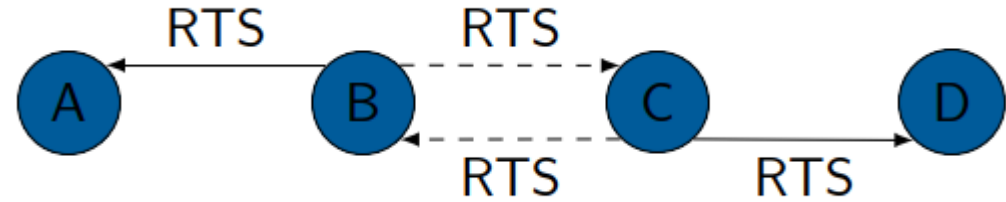


En el siguiente ejemplo, B y C transmiten como **terminales expuestos**:

MACA 4/4

- (1) B y C envían un RTS (Request-To-Send) a A y a D, respectivamente.
- (2) A y D envían un CTS (Clear-To-Send) a B y a C, respectivamente.
- (3) B y C envían las tramas.

En este caso, B y C pueden escuchar el RTS y el CTS de A y D, respectivamente, y transmitir sus propias tramas sin colisiones.



LAN conmutada

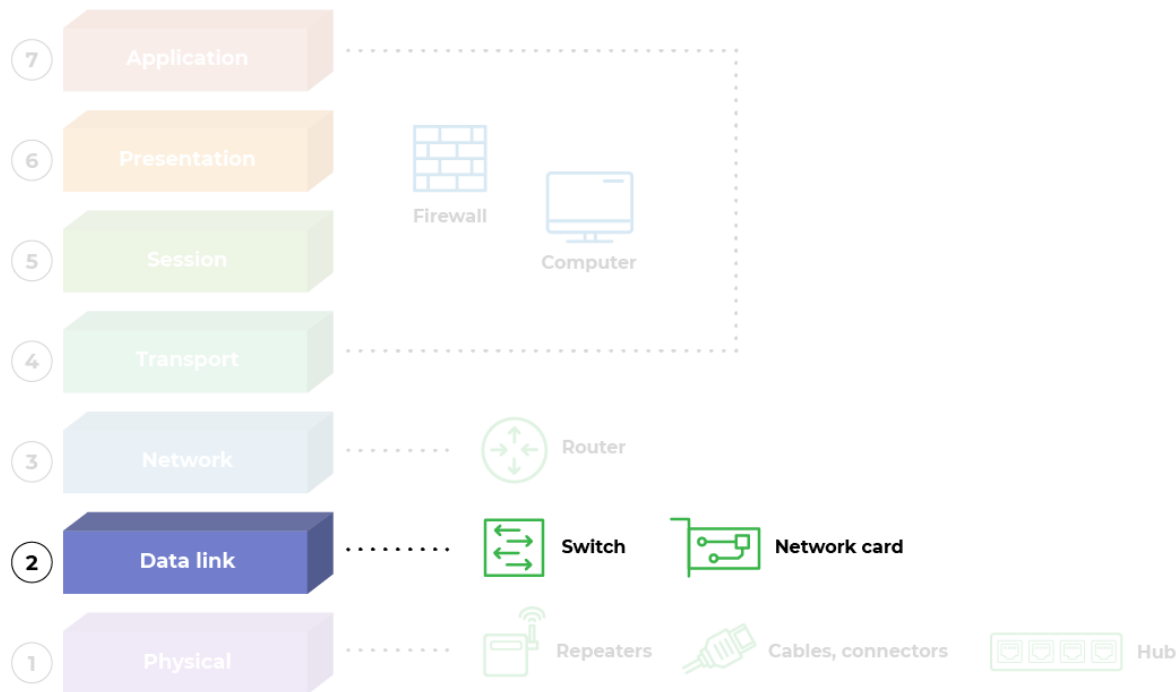
Conmutador

Los concentradores (hubs) y los conmutadores (switches) ofrecen una funcionalidad básica similar: ambos permiten conectar dispositivos en una topología en estrella. A primera vista, pueden parecer idénticos en su apariencia externa. Sin embargo, operan de manera muy diferente en su interior.

Un hub se considera un dispositivo "tonto" que opera en la capa 1 del modelo OSI. Su funcionamiento es simple: si un ordenador en la red envía datos, el hub los transmite a todos los demás dispositivos conectados, sin discriminar.

Por otro lado, un switch es un dispositivo "inteligente" que trabaja en la capa 2 del modelo OSI. Esto significa que es capaz de aprender las direcciones MAC de los dispositivos conectados y almacenarlas en su tabla de direcciones MAC o CAM (Content Addressable Memory). Cuando un ordenador conectado a un switch envía datos a otro dispositivo específico, el switch consulta su tabla de direcciones MAC y envía los datos únicamente al puerto correcto.

Los conmutadores mejoran la eficiencia de la red y reduce los errores. Además, son más seguros. En una red con un hub, cualquier dispositivo conectado puede potencialmente capturar todo el tráfico de la red. Esto es peligroso en caso de que un atacante consiga acceso físico a la red. Sin embargo, con un switch, el tráfico solo se dirige al puerto correspondiente.



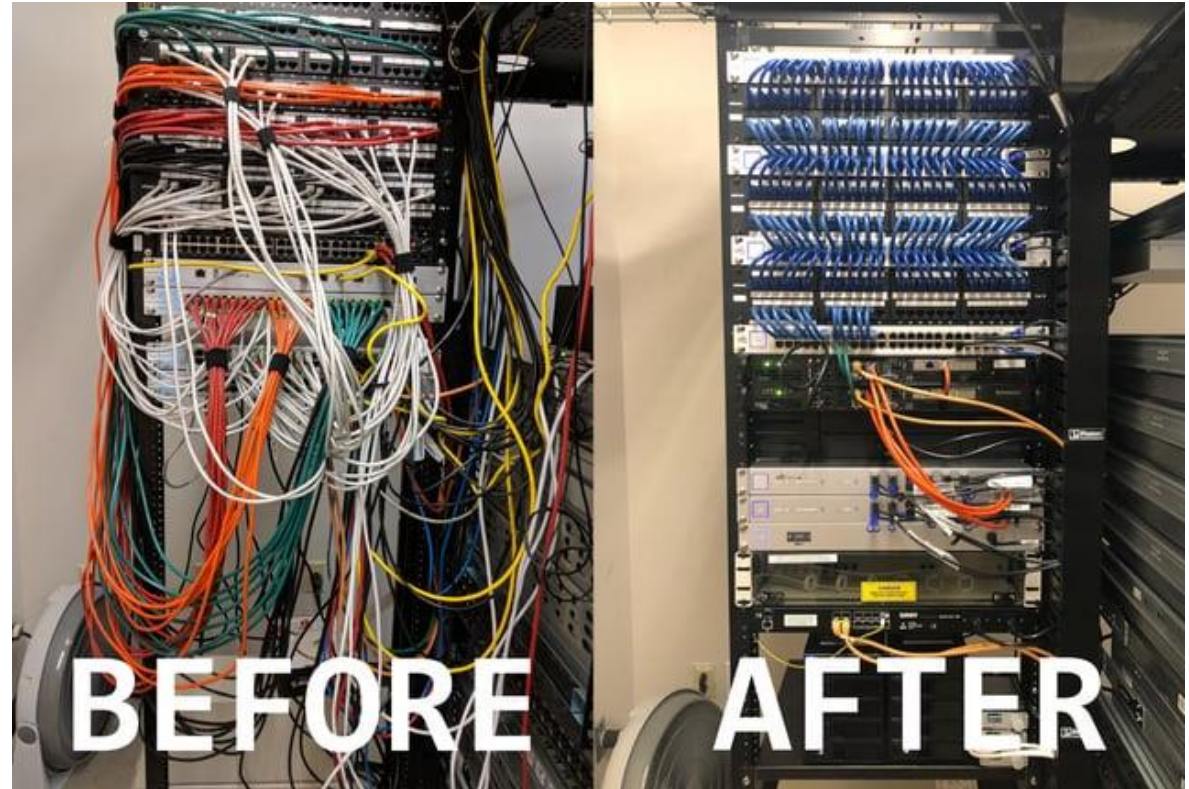
Fuente: See the Big Picture Using the OSI Model <https://openclassrooms.com/en/courses/7414131-set-up-tcp-ip-networks/7603526-see-the-big-picture-using-the-osi-model>

La Ethernet moderna ha evolucionado hacia un sistema más eficiente y seguro, abandonando el uso de un cable compartido por todos los equipos. En su lugar, se utilizan conmutadores (switches) que ofrecen mejores prestaciones, fiabilidad y seguridad.

Los equipos se conectan a los conmutadores LAN mediante cables de par trenzado, que se enchufan en los puertos del conmutador. Estos puertos permiten interconectar los equipos de la red, facilitando la comunicación entre ellos.

Es común que los cables se conecten en armarios (racks) para mantener una mejor organización y facilitar el mantenimiento de la red. De esta manera, la Ethernet moderna ofrece una infraestructura más escalable y flexible para las redes locales.

Ethernet conmutada



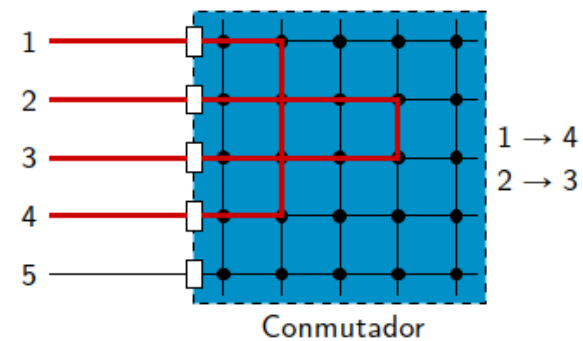
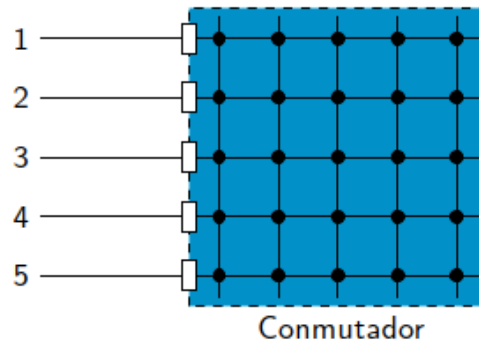
Fuente: Before and after our relay rack reddit.com/r/cableporn/comments/bsjruh/before_and_after_of_our_relay Rack/

En una red Ethernet, cada trama contiene dos direcciones de capa de enlace: la dirección MAC de origen y la dirección MAC de destino. Estas direcciones se utilizan para conectar un puerto de entrada con el puerto de salida correcto, permitiendo que el conmutador (switch) dirija las tramas hacia su destino final.

Una de las ventajas de los conmutadores es que pueden manejar múltiples tramas en paralelo, lo que aumenta la eficiencia y la velocidad de la red. Además, un puerto de conmutador puede ser utilizado tanto para entrada como para salida, lo que permite que varias entradas transmitan al mismo tiempo hacia el mismo puerto de salida.

Sin embargo, es importante destacar que no existe un protocolo de acceso múltiple al medio, lo que significa que no hay un mecanismo para evitar colisiones entre tramas que intentan acceder al mismo puerto al mismo tiempo. Para solucionar este problema, los conmutadores utilizan una zona de memoria llamada buffer o búfer, donde se almacenan temporalmente las tramas en espera de ser transmitidas. Si la sobrecarga en la red fuera sostenida, los buffers pueden llenarse y perderse tramas, lo que puede afectar el rendimiento de la red. Por lo tanto, es importante diseñar y configurar la red de manera que se minimice la congestión y se garantice un flujo de tráfico eficiente.

Conmutadores LAN



Un conmutador necesita encontrar el puerto de salida adecuado para cada trama de entrada. Se busca que los equipos funcionen dentro de la red local sin cambiar configuraciones. Para lograr esto, los conmutadores interpretan las direcciones MAC. Cada trama lleva la dirección MAC del emisor y del destinatario. Los protocolos IEEE 802.xx (como Ethernet, WiFi y Bluetooth) utilizan direcciones MAC de 6 bytes, como por ejemplo 88:E9:FE:80:0E:24 en hexadecimal.

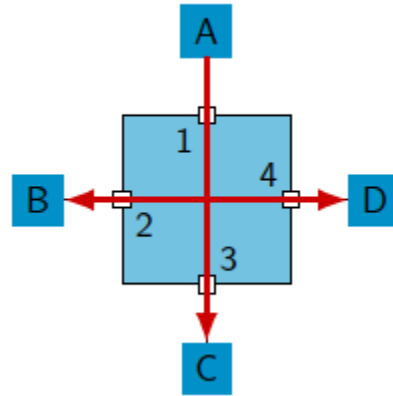
Para funcionar de manera eficiente, los conmutadores utilizan un mecanismo llamado aprendizaje hacia atrás (backward learning), que les permite rellenar una tabla de puertos siguiendo las siguientes reglas:

(1) El conmutador analiza la dirección de origen de las tramas entrantes y asocia el puerto de entrada a la dirección de origen en la tabla

(2) El conmutador verifica si la dirección de destino existe en la tabla. Si existe, pone la trama en el puerto asociado. Si no existe, la envía por todos los puertos (exceptuando aquel del que procede) y espera respuesta.

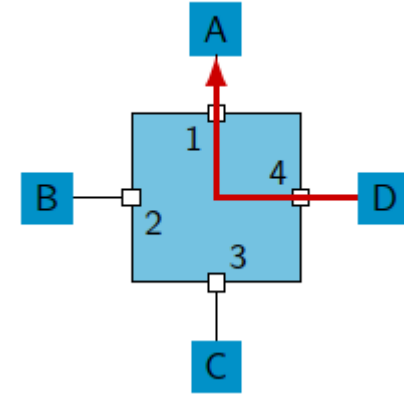
De esta manera, los conmutadores pueden aprender automáticamente la topología de la red y asociar las MAC con sus puertos.

A transmite a D



Puerto	Dirección
1	A
2	
3	
4	

D transmite a A



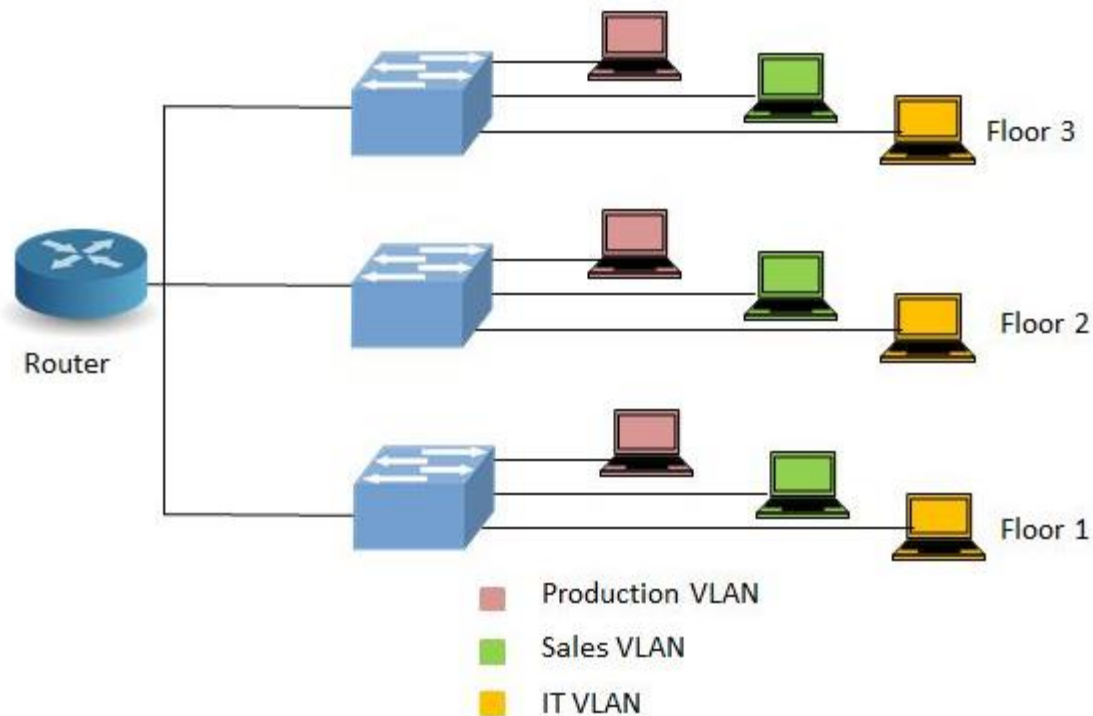
Puerto	Dirección
1	A
2	
3	
4	D

Las Redes LAN Virtuales (VLANs) resuelven el problema de contener el tráfico dentro de grupos de trabajo dispersos geográficamente. Esto permite realizar cambios, agregar y eliminar equipos de manera sencilla y remota, a través de software en una consola. Evita realizar cambios manuales en los cables del armario de conexiones.

El estándar IEEE 802.1Q es el más utilizado actualmente para soportar VLAN en Ethernet. Para definir una VLAN, existen varios mecanismos: (a) VLAN de capa 1, se asigna un puerto físico a una VLAN, puede ser problemático si el nodo cambia de puerto; (b) VLAN de capa 2: se asigna una lista de direcciones MAC a cada VLAN, lo que puede ser complejo de gestionar con muchos nodos; (c) VLAN de capa 3, se utiliza información de la capa de red, como el tipo de protocolo y direcciones de red.

Cada VLAN se identifica con un identificador único llamado VLAN ID (VID), que puede variar de 1 a 4094. Por defecto, todos los puertos pertenecen a la VLAN con VID 1. Mediante el uso de encaminadores (routers), es posible conectar VLAN diferentes, lo que permite una mayor flexibilidad y control en la administración de la red.

Virtual LAN



Fuente: Virtual LAN (VLAN) <https://networkhope.in/virtual-lan/>

¿PREGUNTAS?
andsuarez@uvigo.es



Salvo mención contraria, el contenido de este trabajo está bajo una licencia:

<http://creativecommons.org/licenses/by-sa/4.0/>

